Частное образовательное учреждение высшего образования
«Международный Институт Дизайна и Сервиса»
(ЧОУВО МИДиС)
Высшая школа менеджмента

Кафедра математики и информатики

ДОПУСТИТЬ К ЗАЩИТЕ
Заведующий кафедрой,
к.п.н., доцент
_____С.А. Кондаков
«___» _____2025 г.

ДИПЛОМНЫЙ ПРОЕКТ (РАБОТА)

На тему: «РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ АНАЛИЗА И ВИЗУАЛИЗАЦИИ ДАННЫХ О РОССИЙСКОМ ФОНДОВОМ РЫНКЕ»

Специальность: 09.02.07 Информационные системы и программирование

Руководитель:
преподаватель кафедры
математики и информатики

_____Л.Р. Хаятова

Автор работы:
студент группы П-41

_____К.Е. Жигарь

Челябинск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ ... **3**
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ ... **5**
1.1 Применение веб-технологий в предметной области ... 5
1.2 Анализ существующих решений ... 8
1.3 Бизнес-требования ... 19
1.4 Техническое задание ... 22
ГЛАВА 2. ОБЗОР ТЕХНОЛОГИЙ ... **25**
2.1 Микросервисная архитектура ... 25
2.2 База данных ... 29
2.3 Backend часть ... 34
2.4 Дизайн приложения ... 37
2.5 Frontend часть ... 38
2.6 Виртуальный хостинг ... 40
ГЛАВА 3. РАЗРАБОТКА ПРИЛОЖЕНИЯ ... **46**
3.1 Разработка база данных ... 46
3.2 Backend логика ... 59
3.3 Frontend логика ... 87
3.4 Реализация виртуального хостинга ... 95
ЗАКЛЮЧЕНИЕ ... **101**
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... **104**

ВВЕДЕНИЕ

Современный мир сталкивается с множеством вызовов, требующих комплексного подхода и глубокого анализа. В условиях стремительного развития технологий, глобализации и изменения социально-экономических условий, особую актуальность приобретает тематика создания веб-приложения для анализа и визуализации данных о российском фондовом рынке. Данная работа направлена на изучение фондового рынка ценных бумаг, что позволит выявить ключевые тенденции и проблемы в данной области. Актуальность выбранной темы определяется не только теоретической значимостью, но и практическими аспектами. Она позволяет пользователям уменьшить время и визуализировать данные на фондовом рынки. В условиях сложной и долгой ручной работы необходимо разработать новые подходы и решения, которые могут способствовать упрощению, дополнительной визуализации данных и сокращению работы с анализом российского фондового рынка акций.

Целью данной дипломной работы является разработать и внедрить веб-приложение для мониторинга Российского фондового рынка ценных бумаг, которое предоставляет пользователям инструменты для обеспечения дополнительным удобством в мониторинге и анализа ценных бумаг. Для достижения этой цели будут решены следующие задачи:

1) анализ предметной области;
2) анализ конкурентов;
3) разработка технического задания (ТЗ);
4) выбор технологий и проектирование архитектуры.;
5) разработка макета;
6) реализация backend и frontend;
7) уведомления о значительных изменениях в данных с учётом интеграции приложения в Telegram;
8) парсинг новостей из Telegram-канала;

9) проведение тестирования;

10) оптимизация производительности и исправление выявленных ошибок.

Таким образом, данная работа представляет собой попытку систематизировать знания в области фондового рынка и внести вклад в решение актуальных проблем, стоящих перед инвесторами.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Применение веб-технологий в предметной области

Развитие веб-технологий значительно упростило доступ к финансовой информации, позволив трейдерам и инвесторам получать актуальные рыночные данные в режиме реального времени. В современных условиях анализ фондового рынка требует оперативности и точности, что делает веб-приложения важным инструментом для мониторинга динамики ценных бумаг.

Одним из ключевых направлений применения веб-технологий в сфере финансов является автоматизированный анализ рыночных данных. Веб-приложения позволяют в режиме реального времени обрабатывать и визуализировать данные, полученные с внешних API, таких как открытые интерфейсы банков и брокерских компаний. Это дает пользователям возможность быстро оценивать ситуацию на рынке, отслеживать изменения цен и объемов торгов, анализировать стаканы заявок и прогнозировать движение активов.

Дополнительно веб-технологии обеспечивают интеграцию финансовых сервисов с другими платформами, такими как новостные ресурсы, социальные сети и мессенджеры. Это позволяет трейдерам оперативно получать информацию о событиях, которые могут повлиять на стоимость активов. Например, автоматизированные системы могут анализировать упоминания определенных компаний в новостях и социальных сетях, что помогает инвесторам учитывать рыночные настроения.

Еще одним важным аспектом веб-технологий в трейдинге является персонализация и настройка параметров анализа. Современные веб-приложения предоставляют пользователям возможность выбора индикаторов, настройки отображения данных и использования алгоритмов, которые помогают выявлять торговые сигналы. В сочетании с системой уведомлений это значительно

сокращает время, необходимое на анализ рынка, и повышает точность прогнозов.

Безопасность также играет важную роль в применении веб-технологий в финансовой сфере. Использование двухфакторной аутентификации, шифрования данных и токенов доступа позволяет снизить риски несанкционированного доступа к финансовой информации пользователей, что особенно важно при работе с инвестициями и торговыми платформами.

Для удобного объяснения функция в веб-приложениях по анализам фондового рынка ценных бумаг используется терминология, ниже описаны основные термины, которые будут использоваться в работе.

Форекс – это рынок обмена валюты в международном масштабе [26].

Фьючерсы – при покупке акций компании инвестор приобретает часть её бизнеса. Он может владеть бумагами напрямую или через посредника. Заключение фьючерсного контракта не предполагает реальную покупку актива.

Фондовый рынок – система, в которой осуществляется торговля финансовыми инструментами, такими как акции, облигации, деривативы и другие ценные бумаги. Он играет ключевую роль в экономике, обеспечивая компании возможностью привлекать капитал для развития и инвестирования.

Инвесторы – физические и юридические лица, которые покупают и продают ценные бумаги с целью получения прибыли.

Эмитенты – компании и государственные структуры, которые выпускают ценные бумаги для привлечения капитала.

Брокеры – посредники, которые помогают инвесторам осуществлять сделки на фондовом рынке.

Регуляторы – государственные органы, контролирующие деятельность фондового рынка для обеспечения его стабильности и защиты прав инвесторов [2,3,4].

Опционы – это договор, по которому покупатель может продать или купить актив по заранее установленной цене в оговорённый срок, а может и

отказаться от сделки. Продавец опциона обязуется совершить эту сделку и получает за это деньги [18].

Алготрейдинг – это процесс автоматизированной торговли финансовыми активами, основанный на заранее определённых алгоритмах [14].

Биржевой стакан – это таблица лимитных заявок на покупку и продажу ценных бумаг, контрактов на срочном, товарном или фондовом рынке. Каждая заявка содержит цену (котировку) и количество акций [89].

Индикатор EMA – (экспоненциальная скользящая средняя) – это технический индикатор, который показывает среднюю цену на актив за определённый период времени [90].

Объёмы – это количество акций или других финансовых инструментов, которые были куплены или проданы за определённый период времени. Они показывают активность и интерес инвесторов к определённым ценным бумагам или рынку в целом [25].

Японские свечи – это вид интервального графика и технический индикатор, применяемый для отображения изменений биржевых котировок ценных бумаг [69].

Барные графики – это вид биржевых графиков, который отражает минимальные и максимальные значения цены за определённое время, а также на начало торгов и на момент их закрытия [55].

Линейные графики – это самый простой способ отобразить колебания стоимости акций определённой компании [91].

Хайкен-аши графики – это технический индикатор в виде свечного графика, предназначенный для изучения трендов [80].

Биржевой ордер – это поручение, которое инвестор направляет брокеру для покупки или продажи инструментов на бирже [20].

Трейлинг-стопы – это тип ордера, который позволяет трейдеру максимизировать и защитить прибыль по открытой позиции. Это усовершенствованная форма стоп-ордера, которая автоматически размещает ордер в заданной точке выше или ниже текущей цены [64].

Условные заявки или стоп заявки – это отложенные поручения на покупку или продажу ценных бумаг. В них определены условия, при которых поручение сработает автоматически [15].

One cancels other – это пара условных ордеров, где исполнение одного приводит к автоматической отмене другого [66].

Ренко графики – это тип графиков, который отображает только ценовые изменения [79].

Таким образом, веб-технологии позволяют автоматизировать обработку и анализ финансовых данных, улучшать доступ к информации, повышать персонализацию торговых инструментов и обеспечивать высокий уровень безопасности при работе с финансовыми приложениями.

1.2 Анализ существующих решений

Для того чтобы выяснить какие функции стоит реализовывать для приложения, а какие стоит переработать стоит анализировать текущий рынок похожих приложений. В анализе будут рассмотрены три зарубежных приложений, которые является одними из самых крупных приложения для анализа ценных бумаг. Будут рассмотрены зарубежные программы, потому что на международном рынке представлено большее количество развитых решений, обладающих широким функционалом и передовыми технологиями. Зарубежные приложения зачастую имеют более долгую историю развития, что позволяет выявить наиболее удачные функции, а также определить возможные недостатки, которые можно избежать при разработке собственного продукта.

Кроме того, многие западные платформы обладают удобными инструментами для технического анализа, интеграцией с новостными сервисами и алгоритмическими функциями, что делает их идеальными кандидатами для детального анализа. Сравнение с такими приложениями поможет понять, какие функции наиболее востребованы пользователями и каким образом можно

улучшить собственное приложение, адаптируя его под потребности отечественного рынка.

1. MetaTrader 5 (рис. 1) – это многофункциональная платформа для онлайн-торговли на финансовых рынках, разработанная компанией MetaQuotes Software. Она предназначена для работы с различными финансовыми инструментами, включая форекс, акции, фьючерсы и опционы. MetaTrader 5 (MT5) является преемником популярной платформы MetaTrader 4 (MT4), предлагая более широкий функционал и улучшенные возможности. Данное приложение имеет большой функционал, рассмотрим его основные моменты. MetaTrader 5 поддерживает различные типы графика (линейные, свечные, барные), тем самым не ограничивает пользователя в обзоре данных. Встроено более тридцати индикаторов технического анализа, которые помогают трейдерам проводить глубокий анализ рыночной ситуации и принимать обоснованные торговые решения. Кроме того, платформа предлагает инструменты для алготрейдинга, позволяя пользователям создавать собственные торговые роботы и автоматизировать процессы торговли. Это значительно упрощает работу как для начинающих трейдеров, так и для опытных специалистов, которые стремятся оптимизировать свои стратегии.

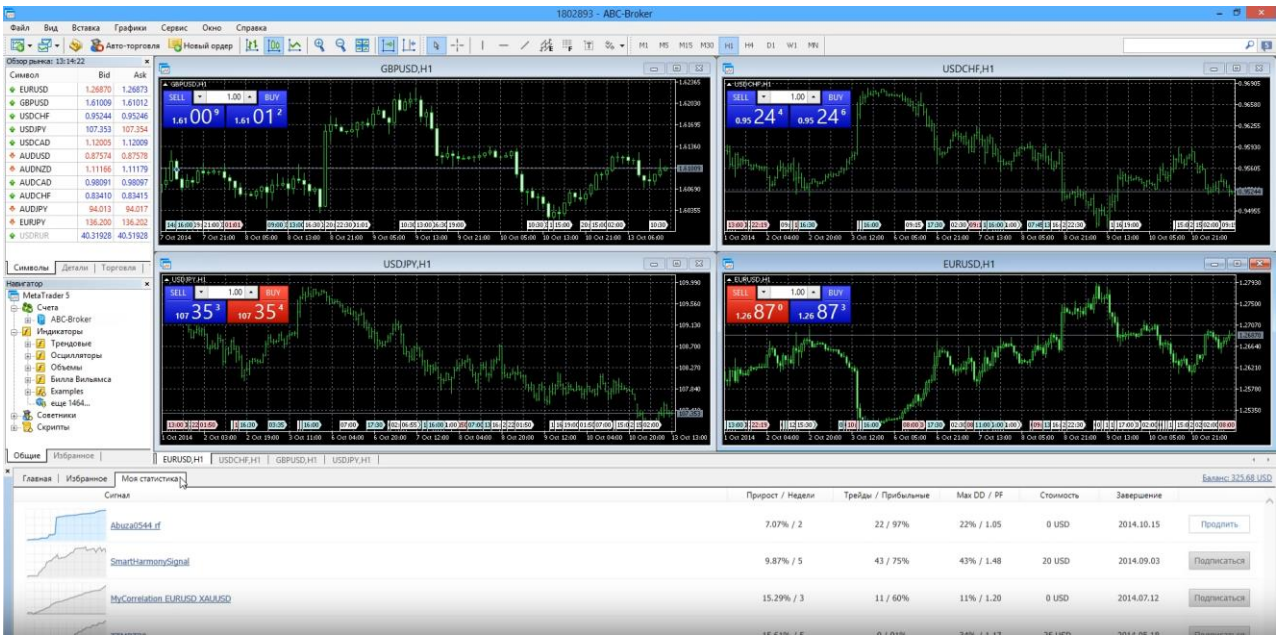


Рисунок 1 – Интерфейс программы MT5

MT5 также включает в себя встроенный экономический календарь, который позволяет трейдерам следить за важными экономическими событиями и новостями, способными повлиять на движение рынка. Мобильная версия платформы предоставляет возможность торговать и анализировать рынок в любое время и в любом месте, что делает её удобной для активных трейдеров. Платформа поддерживает управление несколькими счетами одновременно, что позволяет пользователям эффективно распределять свои активы и управлять рисками. Несмотря на все свои преимущества, MetaTrader 5 может показаться сложной для новичков из-за обилия функций и настроек, а также требует больше системных ресурсов по сравнению с MT4. Однако, благодаря большому сообществу пользователей и множеству обучающих ресурсов, трейдеры могут быстро освоить платформу и использовать её возможности в полной мере. В целом, MetaTrader 5 является мощным инструментом для онлайн-торговли, который сочетает в себе широкий функционал и удобство использования, что делает его популярным выбором среди трейдеров по всему миру.

MetaTrader 5 предлагает множество возможностей для трейдеров, сочетая в себе мощные инструменты анализа и автоматизации. Рассмотрим подробнее ключевые преимущества платформы:

Плюсы:

– программа имеет широкий выбор в индикаторах и множество инструментов для технического анализа, что позволяет трейдерам проводить глубокий анализ рынка;

– предоставляет возможность создания собственных торговых роботов, и автоматизация торговли значительно упрощает процесс для профессиональных трейдеров;

– встроенный календарь помогает трейдерам следить за важными экономическими событиями и новостями, которые могут повлиять на рынок;

– платформа поддерживает различные типы финансовых инструментов, включая форекс, акции, фьючерсы и опционы, что делает её универсальным решением для различных трейдеров.

Несмотря на обширный функционал и удобные возможности для анализа рынка, у платформы есть и некоторые недостатки, которые могут повлиять на пользовательский опыт.

Минусы:

– сложное освоение интерфейса, тем более для новичка, для них может это сбить с толка и причинить больше негатива чем пользы. Можно назвать его устаревшим, некрасивым и неудобным;

– платформа требовательная, требует большого количества ресурса устройства, при высоких нагрузках может сильно зависать;

– несовместимость различных скриптов менжу собой, что может привести к проблемам [5].

MetaTrader 5 является мощной торговой платформой, предоставляющей пользователям широкий функционал для работы на финансовых рынках. Благодаря разнообразным инструментам технического анализа, возможности автоматизации торговли и интеграции с экономическим календарем, платформа остается востребованной среди трейдеров различного уровня подготовки.

Однако, несмотря на все преимущества, у MT5 есть и свои недостатки. Богатый функционал делает платформу сложной для освоения новичками, а требовательность к ресурсам может стать проблемой для пользователей с менее мощными устройствами. Кроме того, несовместимость различных скриптов может ограничивать возможности кастомизации и усложнять работу с платформой.

2. Thinkorswim (рис. 2) – это мощная платформа для технического анализа и торговли на фондовых рынках, разработанная компанией TD Ameritrade. Она предназначена для трейдеров, которым нужен продвинутый инструментарий для торговли акциями, опционами, фьючерсами и валютами. Thinkorswim предлагает широкий функционал, включая глубокий анализ рынка, кастомизированные графики, торговые стратегии и возможность автоматизированной торговли. Платформа поддерживает различные виды графиков: линейные, свечные, барные, линейный прорыв. Это дает возможность

детального анализа ценового движения. В Thinkorswim встроено более 400 индикаторов и инструментов анализа. Пользователи также могут создавать собственные индикаторы с помощью встроенного языка ThinkScript. Платформа предлагает продвинутый анализ объёмов и рыночного стакана Level II. Это помогает трейдерам отслеживать заявки крупных игроков и оценивать ликвидность. Благодаря ThinkScript трейдеры могут создавать алгоритмические стратегии, тестировать их на исторических данных и автоматически выполнять сделки. Thinkorswim интегрирован с новостными сервисами, такими как CNBC и Dow Jones, позволяя получать актуальные финансовые новости в режиме реального времени. Платформа имеет мобильное приложение для Android и iOS, что позволяет трейдерам оставаться в курсе рынка и управлять своими позициями с любого устройства. Хотя Thinkorswim можно использовать и начинающим трейдерам, платформа в большей степени ориентирована на профессионалов из-за высокой сложности интерфейса и обилия инструментов.



Рисунок 2 – Интерфейс программы Thinkorswim

Помимо основных функций, Thinkorswim предлагает расширенные возможности для тестирования стратегий, включая режим PaperMoney, который позволяет трейдерам проверять свои торговые идеи без риска реальных убытков. В платформу также встроена возможность работы с опционами, включая

визуализацию риск-профилей и анализ стратегий. Система поддерживает множество типов ордеров, включая условные заявки, трейлинг-стопы и OCO (One Cancels Other), что делает процесс торговли гибким и удобным.

Еще одной ключевой особенностью Thinkorswim является глубокая кастомизация интерфейса. Пользователи могут изменять расположение окон, настраивать цветовые схемы, создавать кастомные индикаторы и даже использовать встроенный скриптовый язык ThinkScript для автоматизации анализа данных. Это делает платформу мощным инструментом для технических аналитиков и алготрейдеров.

Thinkorswim является одной из самых продвинутых платформ для технического анализа и торговли, предоставляя трейдерам широкий набор инструментов для работы с различными финансовыми активами. Рассмотрим подробнее ключевые преимущества платформы.

Плюсы:

– огромное количество индикаторов и кастомизируемых инструментов – в платформе доступно более 400 технических индикаторов, а также возможность создания собственных инструментов с помощью ThinkScript, что делает её удобной для продвинутых аналитиков;

– гибкость в создании собственных торговых стратегий – пользователи могут разрабатывать и тестировать торговые стратегии, используя встроенный алгоритмический язык, а также настраивать автоматическое выполнение сделок.

– глубокий анализ рынка, стакана и объёмов – Thinkorswim предоставляет расширенные данные Level II, позволяя трейдерам отслеживать поведение крупных игроков и анализировать ликвидность;

– интеграция с новостными источниками – платформа поддерживает потоковую ленту новостей от CNBC, Dow Jones и других сервисов, что помогает трейдерам мгновенно реагировать на важные события.

Несмотря на обширный функционал и гибкость в настройке, платформа имеет ряд недостатков, которые могут повлиять на удобство её использования.

Минусы:

– сложный интерфейс, который может быть трудным для новичков – из-за огромного количества функций и настроек платформа требует значительного времени на изучение и освоение;

– высокие системные требования – Thinkorswim потребляет много ресурсов компьютера, особенно при работе с несколькими графиками и потоковыми данными;

– доступен только через TD Ameritrade – платформа привязана к брокеру TD Ameritrade, что ограничивает её использование за пределами США;

– относительно сложная настройка алгоритмической торговли – несмотря на мощные возможности ThinkScript, начинающим пользователям может быть сложно разобраться в программировании и создании собственных алгоритмов;

– медленный отклик при высокой нагрузке – при одновременном использовании большого количества графиков, производительность может снижаться [9].

В целом, Thinkorswim представляет собой мощный инструмент для профессиональных трейдеров, предоставляя глубокий анализ рынка, гибкость в создании торговых стратегий и широкий выбор индикаторов. Однако высокая сложность интерфейса и ресурсоемкость платформы делают её менее доступной для начинающих пользователей. Кроме того, привязка к брокеру TD Ameritrade ограничивает её использование за пределами США. Несмотря на эти недостатки, платформа остается одним из наиболее функциональных решений для технического анализа и алгоритмической торговли.

TradingView (рис. 3) – это облачная платформа для технического анализа и торговли, предназначенная для трейдеров и инвесторов. Она предоставляет широкий спектр инструментов для работы с акциями, криптовалютами, форексом и другими финансовыми инструментами. Одним из ключевых преимуществ TradingView является его удобный веб-интерфейс, доступный без необходимости установки дополнительного.



Рисунок 3 – Интерфейс программы TradingView

TradingView предлагает более 100 встроенных индикаторов, включая популярные инструменты. Пользователи также могут создавать собственные индикаторы и стратегии с помощью языка Pine Script.

Платформа поддерживает различные типы графиков, включая свечные, линейные, ренко и хайкен-аши. Доступны функции для работы со стаканом заявок, анализа объёмов и потока ордеров.

Одной из сильных сторон TradingView является его социальная составляющая: пользователи могут делиться своими идеями, комментировать прогнозы других трейдеров и подписываться на ведущих аналитиков.

Платформа также интегрирована с брокерами, такими как OANDA, TradeStation и Alpaca, что позволяет вести торговлю прямо из интерфейса TradingView.

TradingView является одной из самых популярных облачных платформ для технического анализа, предоставляя трейдерам удобный интерфейс, широкий выбор индикаторов и возможность социального взаимодействия. Рассмотрим подробнее ключевые преимущества платформы.

Плюсы:

– интуитивно понятный интерфейс – удобный и современный дизайн делает работу с платформой приятной даже для новичков;

– облачная платформа – не требует установки, работает через браузер на любом устройстве;

– широкий выбор индикаторов и возможность их создания – более 100 встроенных индикаторов и кастомизация через Pine Script;

– социальное взаимодействие – возможность делиться идеями и анализировать прогнозы других трейдеров;

– доступность бесплатной версии – базовый функционал доступен бесплатно.

Несмотря на удобство использования и широкие аналитические возможности, платформа имеет ряд ограничений, которые могут повлиять на выбор трейдеров.

Минусы:

– ограниченные функции в бесплатной версии – для продвинутых инструментов требуется подписка;

– задержки данных – в бесплатном доступе рыночные данные могут обновляться с небольшой задержкой;

– ограниченное количество брокеров – платформа не поддерживает всех брокеров, что может быть неудобным для некоторых трейдеров [11].

TradingView – это удобная и гибкая платформа, ориентированная как на начинающих трейдеров, так и на профессионалов. Её облачная структура, интуитивно понятный интерфейс и широкий выбор инструментов для анализа делают её популярным выбором среди инвесторов. Дополнительным преимуществом является развитая социальная составляющая, позволяющая пользователям делиться своими прогнозами и обсуждать рыночные тенденции. Однако ограниченный функционал бесплатной версии, возможные задержки в обновлении данных и поддержка не всех брокеров могут стать препятствиями для некоторых трейдеров. В целом, TradingView остается одним из самых

удобных инструментов для технического анализа благодаря доступности, гибкости и активному сообществу пользователей.

Для наглядного сравнения рассмотренных платформ целесообразно представить их ключевые характеристики в структурированном виде. В таблице 1 приведено сопоставление функционала Thinkorswim, MetaTrader 5 и TradingView, что позволит оценить их преимущества и недостатки с точки зрения использования для технического анализа и торговли.

Таблица 1 – Сравнение существующий решений

Характеристика	Thinkorswim	MetaTrader 5	TradingView
Поддержка рынков	Акции, опционы, фьючерсы, валюты	Форекс, акции, фьючерсы, опционы	Акции, форекс, криптовалюты, фьючерсы
Количество индикаторов	400+	30+	100+
Алгоритмическая торговля	ThinkScript	MQL5	Pine Script
Интеграция с новостями	CNBC, Dow Jones	Встроенный экономический календарь	News, CNBC
Интуитивность интерфейса	Сложный	Сложный	Простой
Глубокий анализ стакана	Есть	Нет	Нет
Системные требования	Высокие	Высокие	Низкие
Бесплатность	Нет, ограниченный доступ	Да	Частично

Анализируя три платформы – MetaTrader 5, Thinkorswim и TradingView, можно выделить их сильные и слабые стороны, а также определить, какие из них наиболее удобны и функциональны для различных категорий трейдеров.

MetaTrader 5 мощная классическая платформа для торговли на финансовых рынках, особенно популярная среди форекс-трейдеров. Она предлагает широкий выбор инструментов для технического анализа и автоматизированной торговли, однако её главной проблемой, с которой не могут смериться многие инвесторы это интерфейс. Он очень сложный, местами не понятный пользователю.

Thinkorswim ориентирована на профессиональных трейдеров, предлагая больше 400 индикаторов, кастомные скрипты ThinkScript, который является большим плюсом для приложения. Продвинутый анализ стакана Level II и интеграцию с новостными сервисами. Однако она требует высоких системных ресурсов, сложна в освоении. Одним из самый серьёзных недостатков приложения является сильно урезанный функционал для инвесторов из стран помимо США, именно этот урезанный функционал заставляет искать другие приложения.

TradingView выделяется простым и интуитивным интерфейсом, большим выбором инструментов технического анализа, социальной составляющей и удобством работы прямо в браузере. Однако у бесплатной версии есть ограничения по количеству индикаторов, а также отсутствие встроенной автоматизированной торговли, что может быть минусом для трейдеров, работающих с алгоритмами.

Если рассматривать платформы по удобству, гибкости и доступности, то TradingView выглядит наиболее универсальным решением. Оно не требует установки, имеет понятный интерфейс, предлагает широкий набор индикаторов и позволяет анализировать графики с любого устройства. Однако для профессиональных трейдеров, особенно тех, кто занимается алготрейдингом или глубоким анализом стакана, лучшим вариантом будет Thinkorswim, несмотря на его сложность.

Если нужно универсальное решение для форекс-торговли, то MetaTrader 5 остаётся хорошим выбором, но его устаревший интерфейс и ограниченные возможности кастомизации делают его менее привлекательным по сравнению с конкурентами.

В итоге можно сказать, что при разработке приложения стоит разработать удобный и интуитивно понятный интерфейс, подобный TradingView, который не перегружен сложными настройками и позволяет быстро освоить платформу. Также важно обеспечить кроссплатформенность и облачную работу, чтобы пользователи могли анализировать рынок и совершать сделки с любого

устройства без необходимости установки программы. Следует внедрить глубокий анализ стакана и объёмов, как в Thinkorswim, чтобы профессиональные трейдеры могли отслеживать ликвидность и действия крупных игроков. Дополнительно необходимо предусмотреть возможность создания и автоматизации торговых стратегий с помощью встроенного скриптового языка, как это реализовано в Thinkorswim и MetaTrader 5, что позволит пользователям разрабатывать собственные алгоритмы и оптимизировать процесс торговли.

Однако при разработке следует найти баланс между богатым функционалом и удобством использования. Также необходимо позаботиться об оптимизации работы платформы, чтобы она не требовала чрезмерных системных ресурсов, как Thinkorswim, и работала стабильно даже при высокой нагрузке. Такой подход позволит создать платформу, сочетающую простоту, функциональность и высокую производительность.

1.3 Бизнес-требования

При разработке приложения для мониторинга российского фондового рынка ценных бумаг необходимо учитывать ряд ключевых бизнес-требований, которые обеспечат удобство работы пользователей, достоверность данных и высокую производительность системы. Основная задача приложения – предоставить трейдерам и инвесторам инструменты для быстрого анализа рыночной ситуации, отслеживания динамики цен и выявления потенциальных торговых возможностей.

Одним из главных инструментов приложения является построение графиков и индикаторов, в частности EMA (экспоненциального скользящего среднего). В базовой конфигурации отображаются EMA 50 и EMA 200, что позволяет отслеживать среднюю цену актива за определенные периоды. Когда EMA 50 (средняя цена за более короткий период) пробивает EMA 200 (долгосрочную среднюю), это сигнализирует о возможном изменении тренда. Такой паттерн часто используется в техническом анализе для определения точек

входа и выхода из позиций. Если краткосрочная EMA пересекает долгосрочную снизу вверх, это может указывать на зарождающийся восходящий тренд, а пересечение сверху вниз – на потенциальное падение цены.

Дополнительным важным инструментом является график объёмов, который позволяет анализировать рыночную активность. Высокие объёмы торговли могут указывать на усиленный интерес участников рынка к активу и предвещать движение цены в одном из направлений. Если объёмы резко возрастают в моменты пробоя ключевых уровней поддержки или сопротивления, это может свидетельствовать о начале сильного тренда.

В приложении также должен быть реализован график стакана за выбранный период. В отличие от большинства торговых платформ, где стакан отображает только текущие заявки, здесь пользователи могут просматривать данные за более широкий временной диапазон. Это позволяет выявлять крупные заявки, появлявшиеся ранее, и анализировать их влияние на цену актива. Стакан отображает текущие заявки на покупку и продажу. Анализ этих данных помогает трейдерам оценивать рыночные настроения и ликвидность актива. Если в стакане фиксируется значительная заявка на покупку, это может свидетельствовать о высоком интересе к активу и возможном росте его цены. Напротив, крупные заявки на продажу могут указывать на давление со стороны продавцов и потенциальное снижение стоимости актива. Такой подход к анализу стакана делает приложение более функциональным и полезным для глубокого понимания рыночной ситуации. Так же в стакане будет реализован список уровней ценной бумаги, этот список с высокой точностью будет показывать пользователям уровни ценны, то есть, когда цена ценной бумаги будет падать, то пользователь увидит до какой конкретной цены скорее всего она упадёт, или наоборот до какой цены вырастит. Такой подход к аналитике ценных бумаг будет являться одним из основных преимуществ приложения.

Для более глубокого понимания ситуации на рынке в приложение будет встроена система анализа новостей, работающая через парсинг Telegram-каналов. Если в информационном пространстве наблюдается рост упоминаний

конкретного актива, это может означать, что вокруг него складывается определённая рыночная ситуация – например, грядут важные события, влияющие на цену. Высокая активность новостей может свидетельствовать о предстоящей волатильности, что делает этот инструмент полезным для принятия торговых решений.

Для удобства пользователей в приложении должно быть предусмотрен список избранных акций, за которыми они хотят следить. В этот список можно будет добавлять или удалять активы в любой момент, а поиск акций будет реализован с поддержкой как полного названия (например, «Газпром»), так и тикера (например, «GAZP»). Это ускоряет доступ к нужной информации и делает работу с приложением интуитивно понятной.

Система уведомлений будет позволять пользователям не пропускать важные рыночные события. Бот в Telegram будет отправлять оповещения о пересечении EMA, резких изменениях объёмов или появлении крупных заявок в стакане. Это позволяет трейдерам своевременно реагировать на движение рынка без необходимости постоянного мониторинга графиков.

С точки зрения безопасности в приложении будет использоваться авторизация с двухфакторной аутентификацией (2FA) через Telegram. Это защищает учетные записи пользователей от несанкционированного доступа и обеспечивает безопасное взаимодействие с системой. Поскольку регистрация пользователей выполняется вручную разработчиками, исключается возможность создания фальшивых аккаунтов, что повышает уровень безопасности приложения.

В итоге приложение сочетает в себе функционал, который делает его удобным и полезным инструментом для трейдеров, позволяя не только анализировать рынок, но и оперативно реагировать на важные события. Использование технических индикаторов, стакана, объёмов и новостного фона в единой системе даёт пользователям комплексное представление о ситуации на рынке и помогает принимать взвешенные инвестиционные решения.

1.4 Техническое задание

Целью разработки является создание приложения для мониторинга российского фондового рынка, предоставляющего пользователям инструменты для анализа рыночных данных, включая графики цен, стаканы заявок, объёмы торгов и новостные события, связанные с активами.

Разрабатываемое приложение направлено на сокращение времени анализа ценных бумаг и повышение эффективности рыночного анализа. Благодаря автоматическому построению ключевых индикаторов, таким как EMA 50 и EMA 200, сбору и обработке данных стакана за выбранный период, а также интеграции с новостными источниками через Telegram, пользователи получают всю необходимую информацию в удобном и наглядном формате. Уведомления о значимых событиях, таких как появление крупных заявок, рост объёмов или частое упоминание актива в новостях, позволяют оперативно реагировать на изменения рынка и принимать более обоснованные торговые решения. Дополнительно предусмотрена возможность поиска акций как по их полному названию, так и по тикеру, что ускоряет навигацию по рынку.

Разрабатываемое приложение ориентировано на активных трейдеров, которым необходим детализированный анализ рыночных данных и оперативное получение уведомлений о значимых изменениях. Благодаря интеграции с Telegram, возможностям гибкой настройки EMA и просмотру исторических данных стакана, приложение станет полезным инструментом для принятия торговых решений. Также, благодаря гибким настройкам параметров анализа, каждый пользователь может адаптировать функционал под свои предпочтения, что делает приложение удобным для широкого круга участников рынка.

Выводы по первой главе

В современном мире финансовых рынков автоматизация анализа данных играет ключевую роль в успешной торговле. Актуальность разработки

приложения обусловлена потребностью в инструментах, которые позволяют трейдерам оперативно получать, анализировать и интерпретировать рыночные данные. Традиционные платформы, такие как MetaTrader 5, Thinkorswim и TradingView, обладают мощным функционалом, но либо перегружены сложными настройками, либо требуют высокой квалификации пользователя. Разрабатываемое приложение ориентировано на снижение времени анализа ценных бумаг и предоставление удобного интерфейса для быстрого принятия торговых решений.

При создании программного продукта был прописан предметный словарь, включающий термины, широко применяемые в трейдинге. Ключевые понятия, такие как EMA, стакан заявок, уровни поддержки и сопротивления, ликвидность, объёмы торгов, являются основными аналитическими инструментами трейдеров и используются в функционале приложения.

Сравнивая разрабатываемое приложение с существующими торговыми платформами, можно выделить несколько преимуществ. В отличие от MetaTrader 5 и Thinkorswim, в приложении упрощён интерфейс и предлагается автоматизированный анализ рыночных данных. В отличие от TradingView, здесь реализована гибкая работа со стаканом, позволяющая просматривать заявки за определённый период, а также инновационный график стакана и возможность получать уведомления о резких изменениях в заявках, объёмах и новостном фоне. Это делает приложение более удобным инструментом для быстрой оценки рынка.

Анализ предметной области показал, что большинство существующих решений ориентированы либо на глубокий технический анализ (как Thinkorswim), либо на графическое представление данных без активных сигналов (как TradingView). Однако для многих трейдеров важно получать оперативную информацию без необходимости вручную настраивать индикаторы и фильтровать данные. Именно эта потребность и легла в основу разрабатываемого продукта.

На основе проведённого исследования была сформулирована структура технического задания, включающая основные требования к функционалу приложения. В нём предусмотрена автоматическая генерация уровней поддержки и сопротивления, мониторинг стакана за конкретный период, пользовательский список отслеживаемых акций, а также система уведомлений о важных рыночных событиях. Дополнительным уровнем безопасности является двухфакторная аутентификация через Telegram-бота, обеспечивающая защиту аккаунтов пользователей.

Таким образом, разрабатываемое приложение предлагает современное решение для эффективного анализа рынка, объединяя автоматический сбор и обработку данных с удобным и интуитивно понятным интерфейсом. Оно позволяет значительно сократить время на анализ рыночной ситуации и оперативно реагировать на изменения.

ГЛАВА 2. ОБЗОР ТЕХНОЛОГИЙ

2.1 Микросервисная архитектура

В процессе разработки программных систем используются различные архитектурные подходы, каждый из которых имеет свои особенности, преимущества и ограничения. Выбор конкретной архитектуры зависит от множества факторов, включая требования к производительности, масштабируемости, отказоустойчивости и удобству сопровождения.

Одной из наиболее распространённых архитектур является монолитная архитектура. В такой системе все компоненты – обработка данных, бизнес-логика и пользовательский интерфейс – объединены в единое приложение. Этот подход хорошо подходит для небольших проектов с ограниченным числом функций. Монолитные системы проще разрабатывать на начальном этапе, так как они не требуют сложной организации взаимодействия между компонентами. Однако с ростом количества функций и пользователей монолитная архитектура начинает терять эффективность. Основные недостатки включают сложность масштабирования, трудности при внесении изменений (из-за высокой связности модулей) и низкую отказоустойчивость (отказ одного компонента может привести к сбою всей системы).

Альтернативой является клиент-серверная архитектура, в которой функциональность разделяется на серверную и клиентскую части. Сервер обрабатывает запросы, управляет данными и выполняет бизнес-логику, а клиент предоставляет интерфейс пользователю. Такой подход позволяет отделить пользовательскую часть от серверной логики, упрощая разработку и сопровождение. Однако при увеличении нагрузки клиент-серверная архитектура может сталкиваться с проблемами масштабируемости, так как сервер остаётся единственной точкой отказа и может перегружаться при росте числа пользователей [13].

Современные крупные проекты всё чаще используют микросервисную архитектуру, в которой приложение состоит из набора независимых сервисов, каждый из которых выполняет строго определённую функцию. Эти сервисы взаимодействуют между собой через API или системы обмена сообщениями, такие как RabbitMQ. Такой подход обладает рядом преимуществ:

- гибкость и модульность – каждый микросервис можно разрабатывать, тестировать и развертывать независимо от остальных компонентов системы;
- масштабируемость – при увеличении нагрузки можно запускать дополнительные экземпляры только тех сервисов, которые требуют увеличенной вычислительной мощности;
- отказоустойчивость – сбой в одном микросервисе не влияет на работоспособность остальных частей системы;
- простота сопровождения – изменения и новые функции можно внедрять без необходимости переработки всей системы [21].

Учитывая вышеуказанные преимущества, при разработке данного приложения была выбрана именно микросервисная архитектура. Основными причинами стали:

1) масштабируемость – необходимость обработки большого объёма биржевых данных в реальном времени, включая работу с графиками, стаканом заявок и анализом новостного фона;

2) гибкость – возможность независимой разработки отдельных сервисов, таких как обработка биржевых данных, уведомления пользователей и анализ Telegram-каналов;

3) отказоустойчивость – снижение риска полного отказа системы при сбоях в отдельных компонентах;

4) асинхронная обработка – для работы с API Тинькоффа требуется высокая скорость обработки запросов, что эффективно реализуется через систему обмена сообщениями RabbitMQ.

Таким образом, выбранная архитектура позволяет создать производительное, надёжное и масштабируемое приложение, способное

работать с большим потоком данных, поддерживать множество пользователей и быстро реагировать на изменения рыночной ситуации.

Микросервисная архитектура разрабатываемого приложения строится на разбиении системы на независимые сервисы, каждый из которых выполняет строго определённую задачу и взаимодействует с другими через API или систему обмена сообщениями RabbitMQ. Разберём микросервисную архитектуру приложения более подробно.

1. Важнейшим элементом системы является база данных, в качестве которой используется PostgreSQL. В ней хранятся данные о пользователях и их настройках, список отслеживаемых ценных бумаг, исторические данные по ценам, объёмам и биржевому стакану, упоминания акций в Telegram-каналах, а также логи событий и уведомлений. База данных взаимодействует с backend-сервисом, который играет центральную роль в системе. Backend обеспечивает доступ ко всей хранимой информации, обрабатывает запросы от frontend, передаёт данные в другие микросервисы и управляет взаимодействием с RabbitMQ. Он также выполняет ключевые операции аутентификации пользователей, в том числе двухфакторную авторизацию через Telegram-бота.

2. Для получения рыночных данных используется отдельный микросервис Manager. Его задача заключается в том, чтобы запрашивать у backend список отслеживаемых акций, формировать на основе этих данных запросы к API Тинькоффа и помещать их в очередь RabbitMQ. Это позволяет контролировать поток запросов и не перегружать внешнее API. После этого за обработку запросов отвечает API Worker, который извлекает их из очереди RabbitMQ, отправляет к API Тинькоффа и получает данные о ценах, объёмах торгов и стакане заявок. Полученные данные затем передаются обратно в очередь RabbitMQ в виде ответов.

3. Следующий ключевой сервис, Handler, отвечает за обработку информации, поступающей из очереди RabbitMQ после работы API Worker. Handler извлекает данные, анализирует их и передаёт в базу данных через backend. Таким образом, данный сервис выполняет роль обработчика и фильтра

полученной информации, обеспечивая корректность и полноту хранимых данных.

4. Помимо сбора биржевых данных, в системе предусмотрен механизм анализа новостного фона с помощью микросервиса Telegram Parser. Этот сервис осуществляет мониторинг выбранных Telegram-каналов, анализирует сообщения на предмет упоминания акций и фиксирует их частоту. Полученные данные отправляются в базу данных через backend, после чего могут быть использованы для дальнейшего анализа и отправки уведомлений пользователям.

5. Telegram Bot является ещё одним важным компонентом системы, выполняющим сразу две ключевые функции. Во-первых, он обеспечивает двухфакторную аутентификацию, проверяя логины и пароли пользователей, а также запрашивая подтверждение входа. Во-вторых, бот используется для оперативного уведомления пользователей о значительных изменениях на рынке, включая появление крупных заявок в биржевом стакане, пересечение индикатора EMA, резкий рост объёмов торгов или частые упоминания акций в Telegram.

6. Взаимодействие с пользователями обеспечивается через frontend-приложение, созданное на основе Next.js. Оно предоставляет удобный интерфейс, позволяющий просматривать графики цен, объёмов и стакана заявок, настраивать параметры EMA и других индикаторов, управлять списком отслеживаемых акций и получать уведомления в реальном времени. Связь frontend с backend реализована через API, обеспечивая быстрый доступ к необходимым данным.

Работа всей системы строится на последовательном обмене данными между микросервисами. Вначале Manager получает список отслеживаемых акций и формирует запросы, которые отправляет в очередь RabbitMQ. API Worker забирает их, делает запросы к API Тинькоффа, получает рыночные данные и помещает их обратно в очередь. Handler забирает полученные данные, анализирует их и передаёт в базу данных. Параллельно Telegram Parser отслеживает новостные упоминания акций и записывает их в базу, а Telegram

Bot уведомляет пользователей о значимых изменениях на рынке. Весь процесс организован таким образом, чтобы минимизировать задержки и обеспечить стабильную работу системы даже при высокой нагрузке.

Использование микросервисной архитектуры даёт приложению ряд преимуществ. Во-первых, оно становится гибким и легко расширяемым, так как новые микросервисы могут быть добавлены без значительных изменений всей системы. Во-вторых, масштабируемость обеспечивается за счёт разделения нагрузки: если число пользователей или объём обрабатываемых данных увеличится, можно просто добавить новые экземпляры API Worker или Handler. В-третьих, система остаётся отказоустойчивой, так как сбой в одном сервисе не приводит к полной остановке работы приложения, а взаимодействие через очереди RabbitMQ снижает зависимость компонентов друг от друга. В-четвёртых, асинхронная обработка данных позволяет эффективно использовать ресурсы и снижает задержки при получении информации.

Таким образом, разрабатываемая микросервисная архитектура обеспечивает эффективный мониторинг российского фондового рынка, удобное взаимодействие пользователей с биржевыми данными и оперативное получение уведомлений о значительных изменениях на рынке. Использование современных технологий позволит создать производительную, масштабируемую и отказоустойчивую систему, способную работать с большим объёмом данных в реальном времени.

2.2 База данных

Выбор базы данных – один из ключевых этапов проектирования системы. В данном случае было решено использовать реляционную СУБД PostgreSQL, поскольку она отвечает требованиям системы по целостности, производительности, расширяемости и надежности.

Все базы данных можно условно разделить на реляционные (SQL) и нереляционные (NoSQL). Каждая из этих категорий имеет свои особенности и применяется в зависимости от структуры данных и характера их использования.

1. Рассмотрим реляционные базы данных (SQL):

– представляют данные в виде таблиц с четко определенной схемой;

– поддерживают отношения между объектами через ключи (Foreign key);

– обеспечивают поддержку транзакций с гарантией целостности (ACID);

– используют мощный язык запросов SQL, позволяющий выполнять сложные аналитические выборки;

– хорошо подходят для структурированных данных с жесткими взаимосвязями, например, финансовых, банковских и учетных систем.

2. Рассмотрим нереляционные базы данных (NoSQL):

– используют гибкую структуру данных, что подходит для хранения JSON-документов, графов, столбцов или ключ-значений;

– не всегда поддерживают строгие транзакции и связи между данными;

– хорошо масштабируются горизонтально, но могут уступать реляционным БД в согласованности данных;

– подходят для больших объемов неструктурированных данных, таких как логирование, работа с соцсетями, кэширование [31].

Важно подметить что Одним из ключевых преимуществ реляционных баз данных является поддержка ACID-свойств, обеспечивающих надежность и целостность данных в системе. ACID – это аббревиатура, обозначающая четыре фундаментальных свойства транзакций: Atomicity (Атомарность), Consistency (Согласованность), Isolation (Изолированность) и Durability (Долговечность). Эти принципы критически важны для финансовых приложений, где ошибки в обработке данных могут приводить к серьезным последствиям.

1. Атомарность (Atomicity) означает, что любая транзакция либо выполняется полностью, либо не выполняется вовсе. Если в процессе выполнения транзакции происходит ошибка, система должна автоматически отменить все изменения, чтобы избежать неконсистентного состояния базы

данных. Например, при переводе средств между счетами банк сначала списывает деньги с одного счета и зачисляет на другой. Если в какой-то момент операция прерывается (например, из-за сбоя сети), система откатывает все действия, не допуская ситуации, когда деньги списаны, но не зачислены.

2. Согласованность (Consistency) гарантирует, что после выполнения транзакции база данных остается в целостном и непротиворечивом состоянии. Это означает, что все бизнес-правила и ограничения (например, уникальные ключи, внешние связи, контроль сумм) должны оставаться неизменными. В финансовых системах, например, не может быть ситуации, когда пользователь переводит больше денег, чем есть на его счету, так как система проверяет баланс перед завершением транзакции.

3. Изолированность (Isolation) определяет, что параллельные транзакции не должны мешать друг другу и приводить к некорректным результатам. Разные уровни изоляции позволяют контролировать, каким образом одна транзакция видит изменения другой. В банковских системах, если один пользователь пытается перевести деньги, а другой в это же время запрашивает баланс счета, изоляция обеспечивает, что пользователь увидит либо старые, либо уже полностью обновленные данные, но не промежуточное состояние.

4. Долговечность (Durability) означает, что после успешного завершения транзакции все изменения надежно сохраняются в базе данных, даже если после этого произошел сбой сервера. Это достигается за счет механизмов журналирования и репликации данных. В случае выхода из строя системы после успешного выполнения транзакции данные не теряются, поскольку они уже записаны в журнал транзакций и могут быть восстановлены.

Поддержка ACID-свойств делает реляционные базы данных надежными и предсказуемыми, что особенно важно для финансовых и биржевых приложений, где любая ошибка может привести к значительным убыткам [12].

В приложении мониторинга фондового рынка данные имеют четкую структуру, требуют строгого соблюдения связей, согласованности и поддержки транзакций, поэтому реляционная база данных является оптимальным выбором.

Рассмотрим выбор базы данных PostgreSQL и сравним её с другими SQL-базами. Наиболее популярными реляционными СУБД являются PostgreSQL, MySQL, MariaDB, Microsoft SQL Server и Oracle Database. Рассмотрим их сравнительные характеристики в соответствии с таблицей 2.

Таблица 2 – Сравнение реляционных баз данных

Характеристика	PostgreSQL	MySQL	MariaDB	Microsoft SQL Server	Oracle Database
Производительность	Высокая (оптимизированный движок, параллельные запросы)	Средняя (менее эффективен при сложных JOIN-запросах)	Хорошая (лучше, чем SQL, но хуже PostgreSQL)	Отличная (но закрытая экосистема)	Отличная (но требует лицензии)
Масштабируемость	Вертикальная и горизонтальная	Вертикальная, ограниченная горизонтальная	Горизонтальная	Вертикальная	Вертикальная
Поддержка ACID	Полная	Частичная	Полная	Полная	Полная
Расширяемость	Высокая (поддержка JSONB)	Сложная	Хуже PostgreSQL	Хуже PostgreSQL	Хуже PostgreSQL
Работа со сложными запросами	Высокая	Средняя	Высокая	Отличная (но закрытая экосистема	Отличная (но требует лицензии)
Стоимость	Бесплатно	Бесплатно	Бесплатно	Платная	Платная

Из этого сравнения видно, что PostgreSQL обладает рядом преимуществ перед конкурентами:

– бесплатность и открытый исходный код – в отличие от Microsoft SQL Server и Oracle, PostgreSQL полностью бесплатен и не требует лицензий;

– продвинутая работа со сложными SQL-запросами – PostgreSQL поддерживает оконные функции, Common Table Expressions (CTE), временные ряды, что делает его идеальным для финансового анализа;

– поддержка ACID и высокой согласованности данных – критично для фондового рынка, где даже небольшая ошибка в данных может привести к серьезным финансовым последствиям;

– гибкость и расширяемость – PostgreSQL поддерживает JSONB, что позволяет комбинировать реляционные и документные структуры, а также легко интегрироваться с другими сервисами;

– масштабируемость – поддерживает как вертикальное, так и горизонтальное масштабирование, что позволяет легко адаптировать его к растущим нагрузкам [72, 30, 70, 71].

Сравнение с NoSQL-базами.

Если бы для проекта использовалась NoSQL-база данных, потенциальными кандидатами могли бы быть MongoDB, Redis и Cassandra. Однако у них есть серьезные недостатки для данной системы:

– MongoDB отлична для хранения документов (JSON), но не поддерживает строгие связи между данными и полноценные транзакции, что может привести к несогласованности информации [29];

– Redis очень быстрая, но предназначена для кэширования и не подходит для хранения больших объемов данных с историей [56];

– Cassandra отлично масштабируется, но ориентирована на большие объемы распределенных данных и не поддерживает сложные SQL-запросы [27].

Таким образом, NoSQL-базы не подходят, так как данные в системе требуют строгой согласованности, транзакций и сложной аналитики.

PostgreSQL была выбрана как основная база данных, поскольку она сочетает в себе высокую производительность, мощные возможности SQL, надежность, бесплатность и гибкость. Это делает её оптимальным решением для финансовых систем, требующих строгой структурированности данных, согласованности и поддержки сложных аналитических запросов.

2.3 Backend часть

Для реализации backend-части системы мониторинга фондового рынка необходимо выбрать язык программирования и фреймворк, обеспечивающие высокую производительность, удобство разработки и поддержку асинхронных операций. Рассмотрим причины выбора Python FastAPI в сравнении с другими популярными фреймворками.

Требования к backend:

1) производительность – API должно обрабатывать большое количество запросов в секунду без задержек и перегрузки сервера;

2) асинхронность – взаимодействие с API брокера (Tinkoff API), телеграм-ботом и базой данных требует эффективного управления потоками;

3) масштабируемость – необходимо, чтобы система могла легко расширяться без переписывания основного кода;

4) простота разработки и поддержки – важно, чтобы разработчики могли быстро вносить изменения и поддерживать код;

5) гибкость интеграции – backend должен без проблем взаимодействовать с PostgreSQL, RabbitMQ, Telegram API;

6) безопасность – система должна быть защищена от SQL-инъекций, XSS, CSRF и других атак.

В качестве языка программирования был выбран Python, так как он обладает широким сообществом, развитой экосистемой и поддерживает асинхронность (async/await). Среди Python-фреймворков рассматривались Django, Flask и FastAPI, но наиболее подходящим для данного проекта оказался FastAPI.

Сравнение Python FastAPI с альтернативами в соответствии с таблицей 3.

Таблица 3 – Сравнение фреймворков

Характеристика	FastAPI	Django	Flask	NestJS	Spring Boot	.NET (ASP.NET Core)
Язык	Python	Python	Python	TypeScript	Java	C#
Производительность	Высокая (близка к Node.js)	Средняя	Средняя	Высокая	Высокая	Высокая
Поддержка асинхронности	Полная (async/await)	Ограниченная (Django Channels)	Ограниченная (через Celery)	Полная (async/await)	Полная (async/await)	Полная (async/await)
Простота разработки	Простая, декларативный синтаксис	Удобен для CRUD-приложений	Прост, но требует дополнительных модулей	Структурированный код, удобен для крупных проектов	Структурированный код, удобен для крупных проектов	Средний уровень сложности
Интеграция с БД	Отличная (SQLAlchemy, PostgreSQL)	Хорошая (Django ORM)	Хорошая (SQLAlchemy)	Хорошая (TypeORM, Prisma)	Хорошая (TypeORM, Prisma)	Отличная (EF Core, SQL Server, PostgreSQL)
Поддержка OpenAPI	Встроенная	Через DRF	Нет	Встроенная	Высокая	Высокая
Масштабируемость	Высокая	Средняя	Средняя	Высокая	Высокая	Высокая

Причины выбора FastAPI:

1) высокая производительность – FastAPI построен на Starlette и Pydantic, что делает его значительно быстрее традиционных Python-фреймворков, таких как Flask и Django. По производительности он приближается к Node.js и Go, что особенно важно для высоконагруженной системы;

2) асинхронность – FastAPI полностью поддерживает async/await, что критично для обработки множества параллельных запросов к API брокера и Telegram. Django и Flask требуют дополнительных модулей для асинхронности, что усложняет разработку;

3) встроенная генерация OpenAPI и документации – FastAPI автоматически генерирует документацию Swagger и Redoc, что упрощает

тестирование API и взаимодействие с frontend-разработчиками. В Django и Flask это требует установки дополнительных библиотек (например, DRF или Flask-RESTful);

4) простота и скорость разработки – Python сам по себе удобен для быстрой разработки. FastAPI минимизирует количество кода, позволяя объявлять эндпоинты в декларативном стиле, что делает его удобным для поддержки;

5) гибкость и интеграция с базами данных и очередями сообщений – FastAPI легко работает с PostgreSQL (через SQLAlchemy), Redis, RabbitMQ (через Celery), что делает его идеальным для микросервисной архитектуры. Django и Flask также могут использовать эти технологии, но требуют дополнительных модулей;

6) безопасность и валидация данных – благодаря Pydantic в FastAPI встроена строгая валидация входных данных, что повышает безопасность приложения;

7) популярность и поддержка – FastAPI активно развивается, его используют крупные компании (Netflix, Uber). В отличие от Flask, который не поддерживает асинхронность "из коробки", и Django, который больше подходит для классических веб-приложений, FastAPI оптимизирован именно для API-разработки [34, 44, 35, 6, 92, 47].

FastAPI был выбран как оптимальный инструмент для backend-разработки, поскольку он сочетает производительность, популярность и поддержка делает разработку конечного продукта проще, асинхронность, удобство разработки и автоматическую генерацию API-документации позволяет быстрее работать с frontend частью приложения. Это делает его лучшим выбором для высоконагруженной системы мониторинга фондового рынка.

2.4 Дизайн приложения

Для проектирования интерфейса приложения была выбрана Figma, так как это отличный онлайн-инструмент, который предоставляет удобные возможности для создания UI/UX-дизайна. Основным преимуществом является облачное хранение, позволяющее получать доступ к проекту с любого устройства без необходимости установки программного обеспечения. Также Figma поддерживает совместную работу, что даёт возможность нескольким дизайнерам и разработчикам одновременно редактировать макет, оставлять комментарии и предлагать правки, это позволит в будущем улучшить дизайн продукта с учётом мнения других людей. Простота использования и интуитивно понятный интерфейс позволяют быстро адаптироваться к инструменту, а функции прототипирования помогают создать интерактивный макет, демонстрирующий пользовательский путь. Дополнительным преимуществом является гибкость, позволяющая легко масштабировать дизайн под различные разрешения экранов [33].

Интерфейс приложения выполнен в тёмной цветовой гамме, что обусловлено несколькими факторами:

1) тёмные темы уменьшают нагрузку на зрение при длительном использовании, что особенно важно в финансовых приложениях, где пользователи анализируют данные на протяжении долгого времени;

2) тёмная тема придаёт интерфейсу профессиональный и современный стиль, что соответствует сфере финансового анализа;

3) высокий контраст между фоном и ключевыми элементами улучшает читаемость информации. В дизайне используются тёмно-синий и чёрный цвета в качестве фона, создающие глубину и не отвлекающие пользователя от данных. Голубые и бирюзовые акценты применяются для выделения важных элементов, таких как графики, заголовки и кнопки, а фиолетовые оттенки используются для дополнительной визуализации данных и повышения их восприятия [48].

2.5 Frontend часть

Для разработки frontend-части системы мониторинга фондового рынка необходимо выбрать фреймворк, который обеспечит высокую производительность, удобство разработки, поддержку серверного рендеринга (SSR), а также возможность масштабирования. Так как приложение связано с динамическими данными, поступающими в реальном времени, важно, чтобы интерфейс загружался быстро, а обновления отображались без задержек.

Фронтенд должен эффективно обрабатывать данные с сервера, взаимодействовать с API брокера, поддерживать пользовательские настройки и обеспечивать удобный пользовательский опыт. Поэтому выбор фреймворка определялся рядом ключевых требований.

Основные требования к frontend:

1. Производительность – важна высокая скорость загрузки страниц, особенно при работе с графиками, таблицами и динамическими данными.
2. SEO-оптимизация – так как приложение ориентировано на финансовые рынки, поисковая индексация страниц повышает его доступность для пользователей [36].
3. Гибкость рендеринга – серверный рендеринг (SSR) и статическая генерация (SSG) позволяют загружать важные данные ещё до рендера на клиенте, что снижает время ожидания.
4. Масштабируемость – фронтенд должен поддерживать возможность расширения функционала без значительных изменений в архитектуре.
5. Простота разработки и поддержки – код должен быть чистым, понятным, с возможностью быстрого внесения изменений и поддержки.
6. Гибкость интеграции – важно, чтобы фронтенд легко взаимодействовал с backend (FastAPI), базой данных (PostgreSQL), брокерским API (Tinkoff) и системой уведомлений (Telegram Bot).

7. Оптимизация загрузки ресурсов – необходимо эффективно управлять статическими файлами, графиками и динамическими компонентами для минимизации нагрузки на клиента.

Среди возможных решений рассматривались Next.js, React (без SSR), Angular, Vue.js и Nuxt.js. Next.js был выбран в качестве основного фреймворка, так как он сочетает в себе преимущества React и предоставляет встроенные возможности для серверного рендеринга, статической генерации страниц и оптимизации загрузки ресурсов.

Был выбран именно Next.js, а не React, потому что в отличие от классического React, который использует клиентский рендеринг (CSR), Next.js позволяет динамически загружать данные на сервере, что делает приложение более быстрым и отзывчивым. Это особенно важно при работе с графиками акций, так как серверный рендеринг снижает нагрузку на клиентское устройство. Также Next.js обеспечивает отличную SEO-оптимизацию, удобную маршрутизацию и поддержку TypeScript, что делает его идеальным выбором для финансового приложения.

Далее приведено сравнение Next.js с другими популярными frontend-фреймворками в соответствии с таблицей 4.

Таблица 4 – Сравнение frontend фреймворков

Характеристика	Next.js	React	Nuxt.js	Angular	Vue.js
Язык	JavaScript, TypeScript	JavaScript, TypeScript	JavaScript, TypeScript	TypeScript	JavaScript, TypeScript
Рендеринг	SSR, SSG, CSR	Только CSR	SSR, SSG, CSR	Только CSR	Только CSR
Производительность	Высокая (SSR+оптимизация)	Средняя	Высокая (аналогично Next.js)	Средняя (сложная структура)	Высокая (аналогично Next.js)
Маршрутизация	Файловая структура	React Router	Файловая структура	Встроенная (модульная)	Vue Router
Поддержка TypeScript	Да	Да	Да	Да	Да
SEO-оптимизация	Отличная (SSR + SSG)	Плохая (только CSR)	Отличная	Хорошая	Средняя
Оптимизация загрузки	Да (автоматическая)	Нет	Да	Нет	Нет

Анализируя представленные, можно сделать вывод, что Next.js является оптимальным выбором для данного приложения по нескольким причинам. В отличие от классического React, он поддерживает серверный рендеринг (SSR) и статическую генерацию страниц (SSG), что улучшает производительность и ускоряет загрузку контента. Это особенно важно для приложения, работающего с динамическими данными фондового рынка, где скорость обновления графиков и таблиц играет ключевую роль.

Другие фреймворки, такие как Angular и Vue.js, также обладают мощными возможностями, но имеют некоторые недостатки в контексте текущего приложения. Angular – более сложен в освоении и требует дополнительного объёма кода для реализации SSR, что увеличивает время разработки. Vue.js, с одной стороны, предоставляет удобный синтаксис и высокую гибкость, но его SSR-решение через Nuxt.js уступает Next.js по производительности и возможностям [46, 84, 40, 45, 17].

Кроме того, Next.js отлично интегрируется с backend на FastAPI, что значительно упрощает разработку и обмен данными между клиентом и сервером. Поддержка TypeScript, автоматическая оптимизация изображений, встроенная маршрутизация и гибкость в рендеринге страниц делают его лучшим выбором для проекта по мониторингу фондового рынка.

Next.js обеспечивает баланс между производительностью, удобством разработки и гибкостью рендеринга, что критически важно для системы, работающей с реальными финансовыми данными.

2.6 Виртуальный хостинг

Для развертывания и поддержки стабильной работы приложения был выбран виртуальный хостинг, обеспечивающий гибкость управления серверной инфраструктурой. В данном проекте виртуальный хостинг реализован с помощью Docker и Docker Compose, что позволило упростить развертывание приложения, его поддержку и масштабирование. В качестве хостинг-платформы

был выбран российский сервис SpaceWeb, серверы которого расположены в Санкт-Петербурге. Этот выбор был обусловлен несколькими факторами:

- юридическая безопасность – использование российского хостинга позволяет избежать возможных ограничений, связанных с хранением и обработкой данных на иностранных серверах;
- минимальные задержки – благодаря размещению серверов в Санкт-Петербурге обеспечивается высокая скорость доступа для пользователей из России;
- отказоустойчивость – SpaceWeb предоставляет надежные серверные мощности с защитой от DDoS-атак и системой резервного копирования;
- гибкость конфигурации – возможность настраивать серверное окружение под конкретные потребности проекта [7].

Однако ключевым элементом развертывания является контейнеризация с Docker, которая позволила создать изолированную, управляемую и масштабируемую среду для работы всех компонентов системы.

Docker – это платформа для контейнеризации приложений, позволяющая запускать программное обеспечение в изолированных контейнерах, которые включают в себя все необходимые зависимости, библиотеки и настройки окружения. В отличие от традиционных виртуальных машин, контейнеры используют ядро операционной системы хост-машины, что делает их быстрее, легче и эффективнее в использовании ресурсов.

Основные преимущества использования Docker в проекте:

1) изолированность сервисов – каждый компонент системы (backend, frontend, база данных, брокер сообщений) запускается в отдельном контейнере, что предотвращает конфликты зависимостей;

2) упрощенное развертывание – для запуска приложения на сервере достаточно одной команды, так как все зависимости уже включены в контейнеры;

3) гибкость и масштабируемость – при увеличении нагрузки можно легко добавить новые экземпляры сервисов без изменения основного кода;

4) кроссплатформенность – контейнеры работают одинаково на разных операционных системах и серверах, что исключает проблемы несовместимости;

5) удобство обновлений – приложение можно обновлять без остановки всей системы, просто заменяя контейнер на новую версию.

Хотя Docker удобен для контейнеризации отдельных сервисов, для работы всей системы требуется одновременный запуск нескольких взаимосвязанных контейнеров. В этом помогает Docker Compose – инструмент для описания и управления мультиконтейнерными приложениями.

С его помощью можно определить конфигурацию всех контейнеров в одном файле (docker-compose.yml) и запускать их одной командой. В данном проекте Docker Compose используется для координации работы следующих сервисов:

- backend (FastAPI) сервис для обработки запросов от клиентов и взаимодействия с базой данных;
- frontend (Next.js) клиентская часть, отображающая данные пользователям;
- database (PostgreSQL) хранение данных о пользователях, акциях и их изменениях;
- rabbitMQ брокер сообщений, управляющий очередями запросов к API;
- API worker и handler сервисы, обрабатывающие данные, полученные от брокера сообщений;
- telegram bot сервис для работы с пользователями через Telegram;
- telegram parser сервис, анализирующий публикации в новостных каналах.

Файл docker-compose.yml содержит информацию обо всех этих сервисах, их зависимостях, параметрах сети и хранилищ данных.

Преимущества использования Docker Compose:

1) единое управление – запуск и остановка всех сервисов осуществляется одной командой;

2) автоматизация – упрощает развертывание системы без необходимости ручной настройки каждого компонента;

3) гибкость конфигурации – позволяет легко изменять параметры сервисов без изменения основного кода [28].

Выводы по второй главе

В данной главе был рассмотрен процесс проектирования и реализации архитектуры системы мониторинга фондового рынка. В ходе анализа были обоснованы ключевые технические решения, обеспечивающие производительность, масштабируемость, отказоустойчивость и удобство поддержки приложения.

Выбор микросервисной архитектуры стал ключевым решением, определяющим гибкость и надежность системы. В отличие от монолитного подхода, микросервисная архитектура позволила разделить приложение на независимые сервисы, каждый из которых выполняет свою функцию: управление пользователями, обработка запросов к API брокера, хранение и анализ данных, работа с очередями сообщений, взаимодействие с Telegram-ботом и парсером новостей. Данный подход обеспечивает масштабируемость системы, так как позволяет независимо изменять и развивать отдельные компоненты без влияния на всю систему.

Выбор реляционной базы данных PostgreSQL обусловлен необходимостью обеспечения целостности и надежности данных. PostgreSQL, обладающий поддержкой ACID-транзакций, обеспечивает строгую консистентность данных, что критично при обработке финансовой информации. В сравнении с NoSQL-решениями PostgreSQL лучше подходит для задач, где требуется сложные реляционные связи и транзакционная обработка данных.

Для backend-разработки был выбран FastAPI, что позволило добиться высокой производительности при минимальном потреблении ресурсов. В отличие от альтернатив, таких как Django или Flask, FastAPI обеспечивает

асинхронную обработку запросов, что особенно важно при интенсивном взаимодействии с API брокера, базой данных и системой уведомлений. Кроме того, FastAPI обладает удобной системой валидации данных, поддерживает OpenAPI и имеет понятный синтаксис, что упрощает разработку и поддержку проекта.

Был рассмотрен дизайн приложения, который играет важную роль в удобстве взаимодействия пользователей с системой. Разработка UI/UX дизайна в Figma позволила создать интуитивно понятный интерфейс, сочетающий функциональность и визуальную эстетику. Были выбраны темные и нейтральные цвета, соответствующие профессиональному стилю финансовых платформ, а также обеспечена адаптивность интерфейса для различных устройств.

В качестве frontend-фреймворка был выбран Next.js, что позволило реализовать серверный рендеринг (SSR) для ускоренной загрузки страниц и лучшей индексации в поисковых системах. В сравнении с React и Vue, Next.js обеспечивает лучшую производительность, встроенный рендеринг на сервере, автоматическую оптимизацию страниц и удобную работу с API. Это позволило значительно улучшить пользовательский опыт и сократить время отклика интерфейса.

Были рассмотрены вопросы виртуального хостинга, который был реализован с использованием Docker и Docker Compose на сервере SpaceWeb. Благодаря контейнеризации система получила гибкость в развертывании, отказоустойчивость и простоту масштабирования. Выбор SpaceWeb, как российского хостинг-провайдера, обеспечил минимальные задержки в передаче данных и соответствие требованиям законодательства.

В результате проектирования и выбора технологий была создана оптимальная архитектура, сочетающая надежность, производительность и удобство эксплуатации. Использование микросервисного подхода, PostgreSQL, FastAPI, Next.js и контейнеризации в Docker позволило реализовать масштабируемую, отказоустойчивую и легко поддерживаемую систему мониторинга фондового рынка.

Каждый этап разработки был направлен на повышение удобства работы пользователей, снижение нагрузки на сервер и обеспечение безопасности системы. В дальнейшем проект может быть легко расширен, например, путем интеграции с новыми API брокеров, добавления аналитических инструментов и автоматизации торговых стратегий.

ГЛАВА 3. РАЗРАБОТКА ПРИЛОЖЕНИЯ

3.1. Разработка база данных

Сущность представляет собой уникальный объект предметной области, который может быть чётко идентифицирован по определённым признакам. Каждая сущность содержит набор атрибутов – характеристик, отражающих важные свойства всех экземпляров этой сущности. Эти атрибуты позволяют не только идентифицировать конкретные объекты, но и описывать их поведение и участие в бизнес-процессах. Для корректной идентификации каждая сущность должна обладать уникальным идентификатором (ключом), обеспечивающим возможность её однозначного различения в рамках системы.

Связь, в свою очередь, – это логическое, осмысленное и формализованное отношение между двумя (или более) сущностями, отражающее наличие взаимосвязи между объектами в реальной предметной области. Каждая связь обладает именем, степенью (один-к-одному, один-ко-многим или многие-ко-многим), а также может быть обязательной или необязательной. Характер связи определяет, каким образом сущности взаимодействуют между собой, и как изменения в одной из них могут повлиять на другую.

Понимание указанных понятий (сущность, атрибут, связь) является критически важным, поскольку инфологическая модель базы данных основывается именно на этих элементах. Инфологическая модель выступает концептуальным представлением предметной области в терминах данных, абстрагируясь от технической реализации. Она служит промежуточным звеном между реальной бизнес-логикой и её будущим воплощением в виде физической структуры базы данных.

Разработка инфологической модели базы данных является важнейшим этапом при проектировании информационной системы. Эта модель формирует основу для построения эффективной, логически обоснованной и устойчивой структуры хранения и обработки информации.

В рамках данной работы инфологическая модель была построена на основе детального анализа бизнес-процессов системы мониторинга фондового рынка. В ходе моделирования были выявлены ключевые сущности, отражающие данные о пользователях, акциях, финансовых показателях и уведомлениях. Каждая сущность дополнена атрибутами, необходимыми для полного и корректного описания информации, актуальной в рамках рассматриваемой предметной области. Также были определены и формализованы связи между сущностями, обеспечивающие логическое взаимодействие данных.

На рисунке 4 представлена диаграмма инфологической модели базы данных, отображающая основные сущности, их атрибуты и взаимосвязи. Данная диаграмма служит важной основой для дальнейшего перехода к логическому и физическому уровням проектирования.

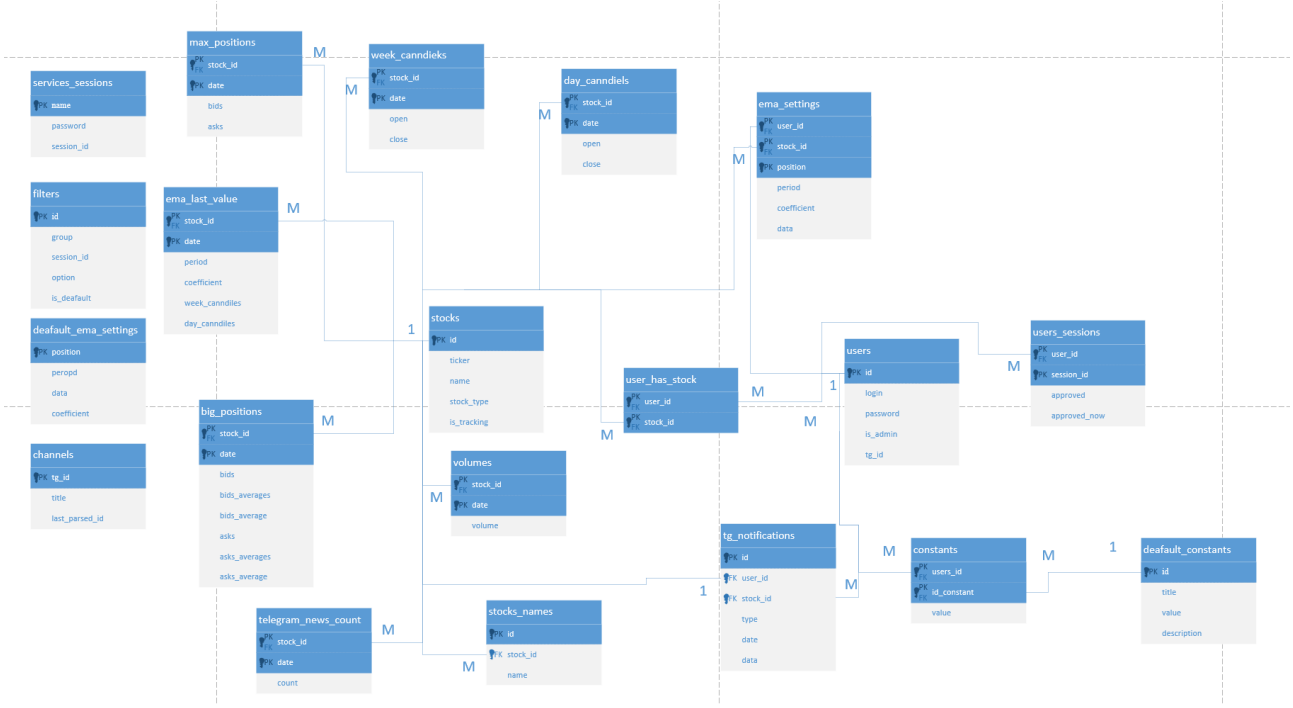


Рисунок 4 – Инфологическая модель базы данных

Инфологическая модель позволила сформировать чёткое представление о структуре данных, что стало основой для построения физической модели базы данных с использованием SQLAlchemy ORM и PostgreSQL. Однако, чтобы добиться оптимального результата, необходимо провести дополнительные исследования и анализ данных, чтобы определить необходимость добавления

или изменения атрибутов и ключей. Подробно опишем сущности информационной модели и связи между ними.

1. Сессии сервисов (services_sessions) – таблица используется для авторизации сервисов, сделано для безопасности, чтобы никто другой не смог подключиться к сервисам. Когда сервисы подключатся, у них появляется уникальный идентификатор сессии для того, чтобы если кто-то узнал логин и пароль, то он не сможет подключиться, так как сервис уже зарегистрирован и используется.

Поля:

– name, уникальное имя сервиса, строчный тип данных;

– password, хешированный пароль, строчный тип данных;

– session_id, уникальный идентификатор, строчный тип данных.

2. Фильтры (filters) – таблица используется для получения информации о фильтрах графиках, чтобы в дальнейшим отобразить данные за нужный период.

Поля:

– id, уникальное имя фильтра, строчный тип данных;

– group, поле отвечает за группу фильтра к которой относиться, например instruments, отвечает за выбор инструмента, чтобы фильтровать ценные бумаги, строчный тип данных;

– option, имя, которое выводиться на frontend, строчный тип данных;

– is_deafault, поле, которое отвечает за изначальный выбранный фильтр, булевой тип данных.

3. Изначальные настройки для (deafault_ema_settings) – таблица используется для отображения изначальных настроек в EMA графике. Эти данные хранятся именно в таблице для быстрой смены данных одним запросом.

Поля:

– position, уникальный идентификатор позиции EMA, строчный тип данных;

– period, за какой период она считается, строчный тип данных;

– coefficient, с каким коэффициентом EMA считается, числовой тип данных;

– data, фильтр открытия или закрытия EMA, строчный тип данных.

4. Изначальные константы (deafault_constants) – таблица используется для отображения изначальных значений в настройках.

Поля:

– id, уникальное имя настройки, строчный тип данных;

– title, короткое название настройки, которое отображается на frontend, строчный тип данных;

– value, значение настройки по умолчанию, которое отображается на frontend, числовой тип данных;

– description, описание настройки, которое отображается на frontend.

5. Пользователи (users) – таблица используется для хранения информации о пользователе, строчный тип данных.

Поля:

– id, уникальный идентификатор, числовой тип данных;

– login, хешированный логин, строчный тип данных;

– password, хешированный пароль, строчный тип данных;

– is_admin, булевое значение, является ли пользователь администратором, булевой тип данных;

– tg_id, уникальный идентификатор пользователя в социальной сети Телеграм, числовой тип данных.

6. Сессии пользователей (users_sessions) – таблица используется для хранения сессий пользователей, поскольку у пользователя, может быть, большое количество сессий с разных устройств.

Поля:

– user_id, уникальный идентификатор пользователя, числовой тип данных;

– session_id, уникальный идентификатор сессии, строчный тип данных;

– approved, булевое поле, подтверждённая ли сессия;

– approving_now, булевое поле, подтверждается ли сессия в данной момент.

7. Константы настроек (constants) – таблица используется хранения значений настроек пользователя.

Поля:

– user_id, уникальный идентификатор пользователя, числовой тип данных;

– id_constant, уникальный идентификатор настройки, числовой тип данных;

– value, значение, строчный тип данных.

8. Ценные бумаги (stocks) – таблица используется для хранения информации обо всех ценных бумаг.

Поля:

– id, уникальный идентификатор, числовой тип данных;

– ticker, сокращённое имя, строчный тип данных;

– name, имя, строчный тип данных;

– stock_type, тип, строчный тип данных;

– is_tracking, булевое значение, отслеживается ли ценная бумага.

9. Пользователи, владеющие акциями (user_has_stock) – таблица используется для хранения акция, которые отслеживают пользователи.

Поля:

– user_id, уникальный идентификатор пользователя, числовой тип данных;

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных.

10. Телеграм уведомления (telegram_notifications) – таблица используется для хранения информации, которую уже отправил, чтобы не повторятся, так же для отслеживания, каким пользователям какая информация была отправлена.

Поля:

– id, уникальный идентификатор, числовой тип данных;

– user_id, уникальный идентификатор пользователя, числовой тип данных;

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– type, тип сообщения, строчный тип данных;

– date, дата, тип данных день, месяц, год;

– data, отправленное значение, строчный тип данных.

11. Настройки EMA пользователя (ema_settings) – таблица используется для хранения значений настроек EMA пользователя.

Поля:

– user_id, уникальный идентификатор пользователя, числовой тип данных;

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– position, позиция, числовой тип данных;

– period, период, строчный тип данных;

– coefficient, коэффициент, числовой с плавающей точкой тип данных;

– data, данные, числовой с плавающей точкой тип данных.

12. Дневные свечи (day_canndiles) – таблица используется для хранения информации о свечах.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– date, дата, тип данных день, месяц, год;

– open, цена на открытие, числовой с плавающей точкой тип данных;

– close, цена на закрытие, числовой с плавающей точкой тип данных.

13. Недельные свечи (week_canndiles) – таблица используется для хранения информации о свечах.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– date, дата, тип данных день, месяц, год;

– open, цена на открытие, числовой с плавающей точкой тип данных;

– close, цена на закрытие, числовой с плавающей точкой тип данных.

14. Последние значение EMA (ema_last_value) – таблица нужна для расчёта EMA для уведомления.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– period, период, числовой тип данных;

– coefficient, коэффициент, числовой с плавающей точкой тип данных;

– date, дата, тип данных день, месяц, год;

– week_canndiles, последнее недельное значение, числовой с плавающей точкой тип данных;

– day_canndiles, последнее дневное значение, числовой с плавающей точкой тип данных.

15. Объёмы ценных бумаг (volumes) – таблица используется для хранения объёмов ценных бумаг.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– date, дата, тип данных день, месяц, год;

– volume, объём, числовой с плавающей точкой тип данных.

16. Большие позиции за день (big_positions) – таблица используется для хранения больших заявок на покупку и на продажу за день, так же на основе этих данных отправляются уведомления.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– date, дата, тип данных день, месяц, год;

– bids, массив максимальных заявок на покупку, которые выше bids_average в формате цена и количество, сточный тип данных;

– bid_average, среднее значение от bids_averages, числовой с плавающей точкой тип данных;

– bids_averages, массив средних значение заявок на покупку, строчный тип данных;

– asks, массив максимальных заявок на продажу, которые выше asks_average в формате цена и количество, строчный тип данных;

– asks_average, среднее значение от asks_averages, числовой с плавающей точкой тип данных;

– asks_averages, массив средних значение заявок на продажу, строчный тип данных.

17. Максимальные заявки за день (max_positions) – таблица используется для хранения максимальных заявок на покупку и продажу ценных бумаг.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– date, дата, тип данных день, месяц, год;

– bids, массив максимальных заявок на покупку в формате цена и количество, строчный тип данных;

– asks, массив максимальных заявок на продажу в формате цена и количество, строчный тип данных.

18. Телеграм Каналы (channels) – таблица используется для хранения данных о Телеграмм каналах, чьи посты обрабатываются через парсинг.

Поля:

– tg_id, уникальный идентификатор Телеграм канала, числовой тип данных;

– title, Название, строчный тип данных;

– last_parsed_id, уникальный идентификатор последнего обработанного поста через парсинг, для того, чтобы этот и предыдущие посты больше не обрабатывать, числовой тип данных.

19. Счётчик упоминаний ценных бумаг за день в Телеграм каналах (telegram_news_count) – таблица используется для хранения данных количества человек, которые увидели упоминаний ценной бумаги за день.

Поля:

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– date, дата, тип данных день, месяц, год;

– count, число человек предположительно видели упоминание ценной бумаги за конкретный день.

20. Псевдонимы ценных бумаг (stocks_names) – таблица используется для хранения псевдонимов ценных бумаг, чтобы парсер алгоритм понимал, какие ключевые слова ему искать.

Поля:

– id, уникальный идентификатор наименования, числовой тип данных;

– stock_id, уникальный идентификатор ценной бумаги, числовой тип данных;

– name, наименование, строчный тип данных.

После детального рассмотрения всех сущностей, входящих в состав инфологической модели, и их атрибутов, важно уделить внимание структуре взаимодействия между этими сущностями. Ведь именно связи формируют

основу логической организации данных и определяют, каким образом информация в системе будет взаимосвязана и согласована.

Связи между сущностями отражают реальные зависимости между объектами предметной области, а также определяют правила доступа, обработки и актуализации данных. Корректное определение связей позволяет не только минимизировать избыточность информации, но и повысить целостность и устойчивость базы данных к ошибкам.

В рамках разработанной инфологической модели были установлены все ключевые связи между таблицами. Эти связи обеспечивают логическую целостность данных и позволяют реализовать сложные запросы, агрегировать информацию и отслеживать динамику процессов в системе мониторинга фондового рынка.

Ниже приведено подробное описание всех связей, сформированных между сущностями в инфологической модели.

1. Связь «один-ко-многим» между сущностями stocks и volumes Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности volumes.

2. Связь «один-ко-многим» между сущностями stocks и big_positions Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности big_positions.

3. Связь «один-ко-многим» между сущностями stocks и max_positions Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности max_positions.

4. Связь «один-ко-многим» между сущностями stocks и telegram_news_count Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности telegram_news_count.

5. Связь «один-ко-многим» между сущностями stocks и stocks_names Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности stocks_names.

6. Связь «один-ко-многим» между сущностями stocks и day_candniles Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности day_candniles.

7. Связь «один-ко-многим» между сущностями stocks и week_candnileks Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности week_candnileks.

8. Связь «один-ко-многим» между сущностями stocks и ema_last_value Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности ema_last_value.

9. Связь «один-ко-многим» между сущностями stocks и ema_settings Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности ema_settings.

10. Связь «один-ко-многим» между сущностями stocks и tg_notifications Образована парой: первичный ключ id из сущности stocks и внешний ключ stock_id из сущности tg_notifications.

11. Связь «один-ко-многим» между сущностями users и tg_notifications Образована парой: первичный ключ id из сущности users и внешний ключ user_id из сущности tg_notifications.

12. Связь «один-ко-многим» между сущностями users и users_sessions Образована парой: первичный ключ id из сущности users и внешний ключ user_id из сущности users_sessions.

13. Связь «один-ко-многим» между сущностями users и constants Образована парой: первичный ключ id из сущности users и внешний ключ users_id из сущности constants.

14. Связь «один-ко-многим» между сущностями default_constants и constants

Образована парой: первичный ключ id из сущности default_constants и внешний ключ id_constant из сущности constants.

15. Связь «один-ко-многим» между сущностями default_ema_settings и ema_settings

Образована парой: первичный ключ position из сущности default_ema_settings и внешний ключ position из сущности ema_settings.

16. Связь «один-ко-многим» между сущностями filters и ema_last_value (предположительно по session_id) Образована парой: первичный ключ session_id из сущности filters и внешний ключ session_id из сущности ema_last_value.

17. Связь «многие-ко-многим» между сущностями users и stocks Образована с помощью связующей таблицы user_has_stock, в которой составной первичный ключ состоит из полей user_id и stock_id, при этом user_id является внешним ключом на users.id, а stock_id – внешним ключом на stocks.id.

После того как определены все сущности и установлены логические связи между ними, следующим этапом проектирования базы данных становится формирование её физической модели. Если инфологическая модель отражает логическую структуру данных и их взаимосвязи на концептуальном уровне, то физическая модель приближена к реальному программному представлению базы данных и фокусируется на технической реализации.

Физическая модель описывает, как данные будут храниться в конкретной системе управления базами данных (СУБД), какие типы данных используются, как реализуются ключи, индексы и ограничения целостности. На этом этапе важную роль начинает играть выбор технологии и инструментов для построения базы данных – в данном проекте используется язык Python в связке с библиотекой SQLAlchemy, реализующей подход ORM (Object-Relational Mapping).

Такой подход позволяет описывать структуру базы данных на уровне объектов языка программирования, что упрощает разработку, сопровождение и масштабирование приложения. Ниже представлено описание физической модели базы данных, её особенностей и связей с программной реализацией.

Физическая модель базы данных отражает реализацию инфологической структуры в конкретной системе управления базами данных – в данном случае PostgreSQL. Она учитывает реальные технические детали хранения данных,

такие как типы данных, индексы, ограничения целостности, первичные и внешние ключи.

В данной системе физическая модель реализована с использованием библиотеки SQLAlchemy – объектно-реляционного отображения (ORM) на языке Python. Это позволяет описывать таблицы базы данных в виде классов, что упрощает взаимодействие с данными и делает структуру проекта более читаемой и масштабируемой.

Каждая сущность из инфологической модели представлена отдельной таблицей в базе данных. Примеры:

– пользователи хранятся в таблице users, содержащей поля: уникальный идентификатор (целое число), логин, пароль, статус администратора (логическое значение) и Telegram ID;

– сессии пользователей описаны таблицей users_sessions, где хранится идентификатор пользователя, идентификатор сессии, флаг одобрения сессии и статус текущего подтверждения. Связь с таблицей пользователей реализована через внешний ключ с каскадным удалением;

– акции хранятся в таблице stocks, где содержатся тикер, название, FIGI-код, тип бумаги и статус отслеживания. Это позволяет системе анализировать как активные, так и потенциально интересные бумаги;

– таблица user_has_stock представляет собой связующую таблицу между пользователями и акциями. Она содержит составной первичный ключ, состоящий из идентификаторов пользователя и акции, и обеспечивает реализацию связи «многие ко многим».

Для всех таблиц задано чёткое определение типов данных: строковые значения, даты, числа с плавающей запятой, логические значения и большие целые числа. Уникальные поля (например, Telegram ID или FIGI) снабжены ограничением уникальности, что предотвращает дублирование информации. Первичные ключи обеспечивают однозначную идентификацию записей, а внешние ключи формируют связи между таблицами, гарантируя логическую целостность данных.

Кроме того, в физической модели применяются индексы для ускорения поиска по важным полям – например, по тикерам или датам. Это существенно повышает производительность при работе с большими объёмами информации, особенно в аналитических модулях системы.

Таким образом, физическая модель системы, реализованная с использованием SQLAlchemy и PostgreSQL, обеспечивает:

- надёжное хранение структурированных данных;
- строгую целостность и согласованность информации;
- поддержку сложных связей между сущностями;
- оптимизированную производительность запросов.

Такой подход полностью соответствует целям и требованиям разрабатываемой системы мониторинга фондового рынка и позволяет эффективно масштабировать проект при необходимости расширения функционала.

3.2. Backend логика

После проектирования структуры базы данных и определения ключевых сущностей следующим логичным этапом стала реализация серверной логики, обеспечивающей надёжное хранение, обработку и передачу информации между клиентом и сервером. Backend играет центральную роль в архитектуре системы, реализуя всю бизнес-логику и гарантируя корректную работу пользовательских функций. На основе спроектированной базы данных были разработаны REST API и механизмы фоновой обработки данных. Это позволило клиентской части запрашивать только актуальные и уже предобработанные сведения. Backend агрегирует данные из нескольких внешних источников, включая финансовые API для рыночных котировок, системы Telegram для сбора упоминаний, новостные агрегаторы и сервисы анализа торговых объёмов. Кроме того, он отвечает за авторизацию и аутентификацию (включая Telegram-верификацию),

управление пользовательскими настройками, расчёт графиков и EMA, работу со стаканами заявок и хранение истории активности.

Архитектура backend построена по микросервисному принципу, где каждый модуль отвечает за отдельный функционал: авторизация, обработка пользовательских данных, аналитика Центральное связующее звено – микросервис, взаимодействующий с базой данных и предоставляющий REST API. Он реализует основные маршруты, среди которых особое место занимает модуль маршрутов /users, отвечающий за все процессы, связанные с сессиями и данными пользователя.

Дополнительно микросервис backend (взаимодействие с базой данных) предоставляет доступ к вложенным маршрутам, и оно разбито по категориям (users, admin, services), каждая из которых включает связанные по логике маршруты. Ниже описание маршрутов, связанных с пользователями.

Модуль __init__.py по маршруту авторизации пользователь предоставляет логин и пароль. Эти данные сначала проходят кодирование (предположительно, для безопасной передачи или хранения), после чего система пытается найти соответствующего пользователя в базе данных. В случае успеха создаётся уникальный идентификатор сессии. Если у пользователя отсутствует Telegram ID, сессия сразу считается одобренной. В противном случае она будет ожидать подтверждения, что может быть связано с реализацией двухфакторной аутентификации. После этого сессия сохраняется в базе и прикрепляется к ответу сервера в виде cookie. Если авторизация не удалась, пользователь получает стандартный ответ об ошибке доступа. Проверка авторизации осуществляется через отдельный маршрут, который извлекает идентификатор пользователя из сессионного идентификатора, находящегося в cookie. Если сессия валидна и одобрена, система возвращает успешный ответ. В противном случае пользователь получает сообщение о том, что он не авторизован или ожидает подтверждения. Для завершения сессии пользователя предусмотрен маршрут выхода. Он также использует куки для получения идентификатора текущей сессии и удаляет соответствующую запись из базы данных. В случае успешного

завершения возвращается стандартный положительный ответ, при возникновении ошибки – сообщение об этом записывается в журнал и возвращается ошибка.

Модуль filters_router реализует маршрут /filters/load_group, который загружает предустановленные фильтры, объединённые в группы. Каждый фильтр содержит идентификатор, описание и признак стандартности. Модуль orders_book_router работает с заявками: маршрут /orders_book/current_price возвращает текущую цену акции, /orders_book/big_positions – крупные заявки по покупкам и продажам, отфильтрованные по масштабу, определяемому пользовательской константой, /orders_book/sums агрегирует объёмы заявок по ценам с округлением, а /orders_book/levels определяет уровни значительных повторяющихся заявок, указывая на возможные зоны поддержки или сопротивления.

Модуль stocks_router.py обрабатывает маршруты, связанные с пользовательским списком акций. Маршрут /stocks/new позволяет добавить тикер: проверяется наличие тикера в базе и ранее выполненное добавление, далее происходит привязка тикера, установка базовых EMA-настроек, активация и сохранение названия. Маршрут /stocks/load загружает все тикеры пользователя и доступные для добавления акции, после чего выполняет модульные проверки (orders_book, tg_news, volumes, emas) с использованием утилит. На основе пользовательских констант выставляются предупреждения, и все тикеры сортируются по количеству активных предупреждений. Маршрут /stocks/delete отвязывает тикер от пользователя по его идентификатору.

Модуль tg_news_router.py реализует маршрут /tg_news/load, загружающий статистику новостей по тикеру. Используя параметры stock_id и group_by, система получает массив новостей за 360 дней и агрегирует данные по неделям, месяцам и дням. Файл volumes_router.py с маршрутом /volumes/load действует аналогично: он получает и агрегирует данные объёмов торгов, не добавляя нулевые значения. Оба маршрута возвращают данные в виде универсального ответа DateCountListResponse.

Модуль constants_router.py предоставляющий доступ к пользовательским константам. Пользователь может загрузить все свои сохранённые константы, каждая из которых содержит название, значение и описание. Кроме того, через этот модуль можно обновлять значение конкретной константы по её идентификатору.

Модуль emas_router.py, отвечающий за работу с экспоненциальными скользящими средними (EMA), которые используются в аналитике и построении графиков. Сначала пользователь может получить настройки EMA для конкретной акции. Эти настройки включают такие параметры, как позиция на графике, период EMA, коэффициент и выбор источника данных. При этом список доступных источников данных также подгружается и отображается вместе с активным значением. Далее предоставляется возможность обновить любую из настроек EMA. Пользователь указывает идентификатор акции, позицию, период, коэффициент и выбранный источник данных. После обновления новая настройка сохраняется в базе. Пользователь может запросить реальные значения EMA по всем своим настройкам для конкретной акции. Система последовательно обрабатывает каждую настройку и рассчитывает значения EMA, группируя их по указанному временному признаку (например, по дням или часам). Результат представляет собой набор точек с датами и соответствующими значениями, пригодный для отображения на графике.

Модуль emas_slow_router реализует маршрут /emas/load, который используется для получения рассчитанных значений, скользящих средних EMA по индивидуальным настройкам пользователя для конкретной акции. Сначала система запрашивает сохранённые пользователем настройки EMA по указанной акции. Затем для каждой настройки вызывается функция расчёта EMA, сгруппированного по выбранному временному интервалу. Особое внимание уделяется позициям: если позиция равна 0, значения добавляются в поле ema_0; если 1 – в ema_1. Это позволяет в результате построить удобную структуру для визуализации, в которой данные сгруппированы по датам с соответствующими

значениями EMA. В случае возникновения ошибки возвращается сообщение об ошибке базы данных.

Модуль orders_book_router занимается анализом биржевых заявок и уровнями поддержки и сопротивления, получая данные из таблиц заявок (order book). Первый маршрут /orders_book/current_price позволяет получить актуальную цену конкретной акции. Второй маршрут, /orders_book/big_positions, фокусируется на крупных позициях в заявках на покупку и продажу за указанный временной диапазон. Здесь используется масштаб, определённый через пользовательскую константу, чтобы отфильтровать только значимые позиции. Результатом являются отсортированные по цене списки заявок на покупку (bids) и продажу (asks), содержащие только крупные заявки. Маршрут, /orders_book/sums, агрегирует суммарные объёмы заявок по ценам за указанный временной интервал. Объёмы покупок и продаж накапливаются по каждой цене, затем округляются в зависимости от минимальной цены в выборке, чтобы сгруппировать близкие значения и сделать анализ более наглядным. Результатом является список сумм по округлённым уровням цен. Маршрут – /orders_book/levels, – выполняет анализ повторяющихся крупных позиций, чтобы определить уровни, где заявок особенно много, что может указывать на потенциальные зоны поддержки или сопротивления. Порог значимости и масштаб выборки определяются на основе пользовательских констант. После вычислений уровни сортируются по проценту значимости и возвращаются с указанием цены, объёма, направления (покупка или продажа) и относительного веса.

Модуль volumes_router.py реализует маршруты для работы с объёмами торгов по тикерам. Префикс маршрутов – /volumes, авторизация обязательна. Маршрут /volumes/load работает по тем же принципам: принимает параметры stock_id и group_by, получает данные объёмов за 360 дней с помощью функции get_volumes, после чего агрегирует их с помощью concat_date_counts. Отличие от предыдущего маршрута – параметр add_zeros=False, который означает, что в результирующем списке не будут добавлены даты с нулевыми значениями

объёма. Как и в случае с новостями, возвращается DateCountListResponse, содержащий список значений по датам.

Таким образом, маршруты из файлов __init__.py, constants_router.py, stocks_router.py, tg_news_router.py, volumes_router.py, emas_router.py, emas_slow_router.py, filters_router.py, orders_book_router.py обеспечивают пользователю полный контроль над акциями, позволяют получать агрегированные данные и строить аналитику на основе новостей, объёмов, EMA и заявок, а также отслеживать предупреждения и фильтры.

Центральную аналитическую роль играет файл utils.py, который реализует подготовку, агрегацию и интерпретацию временных рядов, выявление сигналов из объёмов, новостей, заявок и EMA. В начале определён универсальный класс, представляющий пару "дата – значение", применяемый при работе с новостями, объёмами и EMA. Далее реализованы функции агрегации по времени, которые позволяют группировать данные и заполнять пропуски. Функции анализа новостей и объёмов сортируют данные, проверяют на всплески активности относительно среднего уровня, что сигнализирует о важных изменениях. Анализ стакана позволяет находить заявки, превышающие средние значения, объединять их и определять уровни поддержки и сопротивления. Расчёт EMA выполняется с учётом пользовательских параметров (период, коэффициент, источник данных), а система может выявлять близость линий EMA как потенциальный сигнал. Все проверки объединяются в сводную функцию, определяющую наличие предупреждений. Финальная часть посвящена более глубокому анализу заявок: крупные заявки объединяются по цене, выявляются повторяющиеся уровни, и по частоте их появления система определяет наиболее значимые уровни, которые затем используются при построении торговых стратегий.

Таким образом, utils.py представляет собой аналитический центр системы, соединяющий фундаментальные и технические данные в единую структуру, с помощью которой приложение проводит комплексный анализ, выявляет важные

сигналы и предоставляет пользователю всестороннюю оценку текущей ситуации по каждому активу.

После реализации маршрутов, ориентированных на конечного пользователя, следующим важным этапом стало создание административного интерфейса, обеспечивающего контроль, управление и поддержку всей системы. Административные маршруты предназначены для внутреннего использования и предоставляют расширенный доступ к данным и инструментам модерации. Они позволяют управлять пользователями, их активностью, отслеживать поведение системы, редактировать конфигурации и поддерживать стабильную работу проекта. В отличие от пользовательских маршрутов, API для администратора не доступны через основной пользовательский интерфейс и требуют соответствующего уровня доступа.

Модуль __init__.py внутри административного пространства маршрутов (/admin) отвечает за организацию доступа и маршрутизацию к различным подсистемам, обеспечивая централизованную точку входа для администраторов. Все административные маршруты защищены базовой HTTP-аутентификацией, при которой логин и пароль пользователя кодируются в JWT-формате и проверяются на наличие прав администратора. В случае некорректных данных или отсутствия административных полномочий пользователь получает отказ в доступе с соответствующим HTTP-статусом. Помимо обычной проверки прав, реализована дополнительная проверка на статус суперпользователя, которая используется в ряде чувствительных операций. Внутри основного административного роутера /admin последовательно подключаются вложенные маршруты: users_router, services_router, constants_router, filters_router и emas_router, каждая из которых отвечает за свою функциональную область. Это позволяет логически изолировать зоны ответственности и упростить сопровождение кода.

Модуль constants_router реализует маршруты для управления системными константами, которые используются в пользовательских стратегиях и при вычислении сигналов. Маршрут /default_constants/load возвращает актуальный

список всех стандартных констант, включающий их идентификаторы, заголовки, значения и описания. Это базовый инструмент администратора для мониторинга текущей конфигурации системы. С помощью маршрута /default_constants/new можно создать новую константу: она не только сохраняется в системной базе, но и автоматически клонируется для каждого пользователя, чтобы обеспечить целостность пользовательской среды. Изменение параметров существующих констант разбито на три специализированных маршрута – /update_title, /update_value и /update_description – каждый из которых обновляет одно конкретное поле по заданному идентификатору. Удаление константы осуществляется через маршрут /delete, что позволяет администратору исключить устаревшие или ошибочные значения из системы.

Модуль emas_router реализует административные маршруты для управления дефолтными параметрами скользящих средних EMA, которые используются в анализе рыночной динамики. Маршрут /emas/load возвращает список всех предустановленных настроек EMA, каждая из которых включает позицию в графике (порядок отображения), период усреднения, коэффициент сглаживания и источник данных. Источники данных представлены в виде фильтров, объединённых в группу ema_data, и для каждой настройки указывается, какой фильтр используется по умолчанию. Это обеспечивает гибкость настройки поведения EMA по отношению к разным видам ценовых данных – например, закрытию, объёму или объёмно-взвешенным значениям. Обновление существующих настроек осуществляется через маршрут /emas/update, который принимает новый набор параметров и обновляет соответствующую запись в базе. Все операции сопровождаются логированием, а ошибки обрабатываются централизованно.

Модуль filters_router управляет административными маршрутами, связанными с предустановленными фильтрами. Маршрут /filters/load формирует сгруппированный по категориям (группам) список всех фильтров, где каждая группа содержит набор фильтров с указанием идентификатора, текстового значения и признака использования по умолчанию. Это позволяет удобно

управлять фильтрами как в пользовательском, так и в системном контексте. Добавление нового фильтра осуществляется через маршрут /filters/new, при этом предварительно проверяется уникальность фильтра в рамках группы. При необходимости можно изменить опцию фильтра (/update_option) или его статус по умолчанию (/update_is_default). Для удаления используется маршрут /delete, позволяющий удалить фильтр по его идентификатору. Все маршруты поддерживают детальную обработку ошибок и предоставляют унифицированные ответы.

Модуль stocks_router предоставляет административный маршрут /stocks/new, предназначенный для привязки тикера к конкретному пользователю. При вызове осуществляется поиск тикера в базе и проверка на его наличие у пользователя. Если тикер найден и ранее не был привязан, происходит его ассоциация с пользователем. Такая функциональность необходима для администраторов, чтобы вручную добавлять нужные бумаги к аккаунтам, например, при решении спорных ситуаций или в процессе пользовательской поддержки. Все обращения к маршруту защищены авторизацией и журналируются для последующего аудита.

Модуль services_router предоставляет административный интерфейс для управления внешними сервисами, взаимодействующими с системой. Маршрут /services/load_all загружает список всех зарегистрированных сервисов, включая их имя, пароль и текущий session_id. Эти данные позволяют отслеживать активные подключения и обеспечивать безопасность межсервисного взаимодействия. Добавление нового сервиса осуществляется через маршрут /services/new, где необходимо указать уникальное имя и пароль. Сервис автоматически регистрируется в системе и может быть использован для авторизованного доступа. При необходимости можно удалить активную сессию конкретного сервиса с помощью маршрута /services/delete_session, что позволяет принудительно сбросить авторизацию и инициировать повторную аутентификацию. Полное удаление сервиса реализовано через маршрут /services/delete, после чего сервис теряет все права на взаимодействие с системой.

Все действия сопровождаются логированием и защищены от ошибок централизованной системой обработки исключений.

Модуль users_router управляет административными действиями, связанными с пользователями системы. Все маршруты защищены зависимостью is_user_super_user, что ограничивает доступ только для суперпользователей. Основной маршрут /users/load позволяет загружать список пользователей с пагинацией, включая их логин, пароль (декодированные из защищённого формата), административный статус и Telegram ID. Это облегчает аудит и администрирование пользователей. Проверка существования логина выполняется маршрутом /users/is_login_exists, в котором логин шифруется и проверяется на уникальность в базе. Регистрация нового пользователя осуществляется через маршрут /users/new, принимающий логин, пароль и флаг администратора. Если логин уже существует, возвращается сообщение об ошибке. В противном случае создаётся новая запись пользователя, и к нему автоматически привязываются все дефолтные константы системы. Удаление пользователя выполняется через маршрут /users/delete, при этом логин предварительно шифруется. Все действия логируются, а в случае сбоев возвращаются стандартные сообщения об ошибках.

Таким образом, административные маршруты обеспечивают полный контроль над управлением внешними сервисами и пользователями. Они включают операции по созданию, обновлению и удалению записей, применяют уровни доступа, шифрование данных и централизованную обработку ошибок, что гарантирует безопасность и надёжность администрирования всей системы.

Сервисные маршруты, представленные в данном модуле, образуют защищённый backend-интерфейс, предназначенный для работы внешних сервисов с системой. Общий префикс /backend объединяет их в единую структуру, а взаимодействие регулируется сессионной аутентификацией на основе cookie session_id. Главный файл __init__.py содержит маршрут /login, который используется для авторизации сервисов. При успешной проверке имени сервиса и пароля, и при отсутствии уже активной сессии, системе генерируется

уникальный session_id, сохраняемый в базе и устанавливаемый в cookie ответа. Все дальнейшие маршруты требуют валидного session_id и реализуют защиту через зависимость get_session_data, проверяющую его наличие и действительность.

Модуль notifications_router.py реализует функционал по генерации уведомлений для пользователей на основе анализа рыночной активности. Единственный маршрут /notifications/volumes_scales служит для расчёта масштабов объёмов торгов по каждому активному инструменту. Система загружает данные по объёмам за последние 30 дней и сравнивает текущие значения с усреднёнными. Если объём текущего дня превышает месячный средний, рассчитывается коэффициент масштаба. Затем для каждого пользователя, обладающего соответствующим активом и имеющего Telegram ID, производится проверка: если масштаб превышает персональный порог VOLUMES_SCALE_TO_WARNING, и с учётом шага предупреждения VOLUMES_WARNING_STEP событие считается значимым, то либо обновляется существующее уведомление, либо создаётся новое. В результате формируется список сообщений, предназначенных для отправки через Telegram, содержащих краткую информацию по активу и масштаб превышения. Этот механизм обеспечивает точечную, чувствительную к индивидуальным настройкам, нотификацию пользователей при подозрительной торговой активности.

Маршруты модуля sessions_router.py ориентированы на управление пользовательскими сессиями, проходящими через процесс подтверждения. Эндпоинт /sessions/load_unapproved загружает список неподтверждённых сессий и одновременно помечает их флагом approving_now, исключая повторное рассмотрение. Каждая сессия содержит идентификаторы пользователя, сессии и Telegram. Далее, маршрут /sessions/process позволяет либо утвердить сессию (при approved=True), либо удалить её, если она отклонена. Дополнительно, маршрут /sessions/login_with_tg реализует возможность привязки Telegram ID к пользователю на этапе входа: пользователь указывает логин и пароль, они

шифруются, и в случае их корректности проверяется, есть ли уже привязка Telegram. Если нет – осуществляется привязка, и пользователь получает возможность получать уведомления в Telegram. Все маршруты сопровождаются централизованной обработкой ошибок и логированием.

Модуль stocks_router.py реализует набор маршрутов для взаимодействия с данными о фондовых активах в рамках API-сервиса на FastAPI. Все маршруты сгруппированы под префиксом /stocks и защищены зависимостью Depends(get_session_data), что предполагает авторизацию или наличие сессионных данных. Маршруты разделены по функциональным блокам, и каждый из них выполняет строго определённую задачу. Маршрут stocks/load_active возвращает список всех активных акций, которые находятся в отслеживании. Запрос обращается к БД через метод load_active_stocks(), а затем сериализует каждый результат в схему StockModel, включающую ключевые поля – id, ticker, figi, название, тип акции и флаг отслеживания. Ошибки логируются, а в случае проблем с БД возвращается сообщение об ошибке. Маршрут /stocks/is_exists проверяет существование акции по тикеру. В случае, если акция найдена, в ответе возвращается payload=True, иначе – False. Маршрут /stocks/new служит для создания новой записи об акции. Обязательные параметры включают тикер, FIGI, название и тип бумаги. Дополнительно можно указать текущую цену, объём, флаг отслеживания и необходимость создания имени (create_name), что создаст дополнительную запись с названием, сформированным из тикера. Если акция уже существует, возвращается сообщение об ошибке. Маршруты /stocks/update_day_candle и /stocks/update_week_candle обновляют или создают записи по дневным и недельным свечам соответственно. Эти методы принимают параметры тикера, даты, цены открытия и закрытия, а также булевый флаг create, разрешающий создание новой свечи, если она не найдена. Если create=False и свеча отсутствует – операция завершается ошибкой. Далее следует блок, связанный с торговыми объёмами. Маршрут /stocks/new_volume позволяет создать запись объёма торгов за определённую дату, а /stocks/update_volume – обновить объём либо создать

новую запись, если включён флаг create. Проверки на существование записи ведутся через методы базы данных is_volume_exists, и в случае дублирования возвращается соответствующее сообщение. Маршрут /stocks/volumes_count возвращает количество записей объёма по заданному тикеру. Сначала извлекается stock_id по тикеру, далее вызывается метод get_volumes_count. Самый объёмный маршрут – /stocks/update_positions отвечает за обновление данных по крупным и максимальным позициям (ордерам) по акциям на текущую дату. Обработка начинается с получения текущих записей по крупным (big_positions) и максимальным (max_positions) позициям. Если они отсутствуют – создаются новые пустые записи. После этого обрабатывается тело запроса, содержащее массивы bids, asks, и средние значения bids_average, asks_average. Средние значения добавляются в накопленные списки и пересчитываются. Далее обновляются записи в базе. Новые заявки (ордера), чьи объёмы превышают среднее значение, добавляются в список, либо обновляют существующие по совпадению цены. Аналогично обрабатываются максимальные значения: при совпадении по цене фиксируется максимум объёма, либо создаётся новая запись. В завершение маршрут обновляет все агрегированные значения в базу данных и возвращает стандартный положительный ответ. Все возможные ошибки на протяжении выполнения логируются, и в случае сбоев клиент получает унифицированное сообщение о проблемах с базой данных.

Модуль telegram_router.py реализует маршруты FastAPI, относящиеся к телеграм-интеграции и обслуживающие связанные с этим сущности: телеграм-каналы, названия акций и счётчики новостей. Все маршруты сгруппированы под префиксом /telegram с тегом Backend Telegram, и зависят от сессии пользователя через Depends(get_session_data). Первый блок маршрутов отвечает за управление телеграм-каналами. Метод /channels/load_active возвращает список всех активных телеграм-каналов, обернутых в модель TgChannelsListResponse. Каждый канал представлен объектом TgChannelModel, содержащим ID канала (tg_id), его заголовок (title) и идентификатор последней распарсенной записи (last_parsed_id). Далее маршрут /channels/is_exists позволяет проверить,

существует ли телеграм-канал по заданному tg_id, возвращая BaseResponse с логическим флагом в payload. Добавление нового канала реализуется через /channels/new, где передаются параметры tg_id, title и опциональный last_parsed_id (по умолчанию – 0). При наличии канала в базе будет возвращена ошибка с сообщением Channel already exists. Обновление last_parsed_id производится с помощью /channels/update_last_parsed_id, и если канал не существует – возвращается сообщение об ошибке Channel does not exist. Второй блок маршрутов обрабатывает работу с названиями акций. Метод /stocks_names/load возвращает агрегированный список названий акций по тикерам. Каждая уникальная пара stock_id и ticker собирается в модель StockNamesModel, где в списке names аккумулируются все синонимы/названия. Проверка существования конкретного названия по тикеру и имени осуществляется через /stocks_names/is_exists. Если тикер отсутствует в системе, маршрут вернёт ошибку No such ticker. Создание нового названия для акции производится через /stocks_names/new, где требуется stock_id и строка name; при попытке добавить дублирующее название – вернётся ошибка Stock name already exists. Третий блок маршрутов относится к учёту количества новостей по акциям. Метод /news_counters/new позволяет создать новую запись с количеством новостей на текущую дату, при условии, что счётчик ещё не существует. В противном случае возвращается ошибка Today counter already exists. Получение всех текущих новостных счётчиков реализовано в /news_counters/load, где возвращается список моделей TgNewsCounterModel, каждая из которых содержит stock_id и количество новостей (count). Обновление счётчика выполняется через /news_counters/update, и, если счётчик по заданному stock_id отсутствует, возвращается сообщение об ошибке Today news counter do not exists.

Каждый маршрут защищён от сбоев с помощью конструкций try-except, при этом ошибки логируются через модуль log, а пользователю возвращается унифицированный ответ BaseResponse с полем error=True и пояснением Error with database.

В целом, данный блок сервисных маршрутов реализует защищённую аутентификацию сервисов, сбор уведомлений по критическим метрикам объёмов, а также управление пользовательскими сессиями с акцентом на безопасность и прозрачность Telegram-интеграции. Каждая точка входа в систему тщательно валидируется, взаимодействие структурировано и адаптировано под интеграцию с внешними каналами уведомлений.

Одним из важных микросервисов backend-системы является telegram_parser, отвечающий за мониторинг упоминаний акций в Telegram-каналах. Его основная задача – выявлять, какие ценные бумаги обсуждаются в тематических Telegram-сообществах, подсчитывать просмотры и вести аналитику на основе этих данных.

В итоге, микросервис backend представляет собой обособленную серверную часть, предназначенную для взаимодействия с базой данных и организации логики доступа к данным через четко структурированные маршруты. Он условно разделён на три ключевых блока:

– User-маршруты, обеспечивающие доступ к данным конечным пользователям. Здесь обрабатываются запросы, связанные с отображением информации, проверкой наличия данных, загрузкой списка объектов и прочими пользовательскими операциями, направленными на чтение и валидацию информации.

– Admin-маршруты, предоставляющие интерфейс для управления и модификации внутренней структуры данных. Через эти маршруты можно создавать, изменять и удалять записи, а также управлять пользователями и доступами. Это административный слой, необходимый для гибкой настройки системы.

– Service-маршруты, такие как telegram_router.py, реализуют вспомогательные функции, в том числе интеграцию с внешними источниками. В данном случае – с Telegram-каналами. Здесь находятся эндпоинты для управления каналами, загрузки и обновления связанных с ними данных, работы с новостными счётчиками и названиями акций, используемыми в системе. Эти

маршруты играют ключевую роль в организации бэкграунд-взаимодействия и автоматизации процессов.

Таким образом, микросервис backend выполняет роль центрального связующего звена, обеспечивающего как административный, так и прикладной доступ к данным, а также поддержку сервисных задач.

Следующим логическим элементом архитектуры является микросервис telegram_parser. Он предназначен для асинхронной обработки и парсинга информации из телеграм-каналов. Основная задача данного микросервиса – извлекать данные из указанных источников, фильтровать, обрабатывать и передавать полученную информацию в backend для последующей обработки и хранения. Ниже мы подробно рассмотрим его внутреннюю структуру, ключевые компоненты и взаимодействие с другими частями системы.

Основные функции сервиса:

– аутентификация с backend-сервером. Сервис использует служебный аккаунт, имя и пароль которого хранятся в .env-файле. После авторизации сервис получает cookies, которые используются для дальнейшего обмена данными с основным API;

– получение списка отслеживаемых тикеров. Перед началом работы сервис делает запрос к API backend-системы, чтобы получить актуальный список тикеров (акций) и их синонимов, упоминание которых он должен искать в Telegram-сообщениях;

– загрузка списка активных Telegram-каналов. Далее сервис получает из backend-API список всех каналов, которые ранее были добавлены системой. При необходимости сервис может добавить новый канал автоматически, если он найден в списке диалогов;

– парсинг сообщений. С помощью библиотеки telethon осуществляется подключение к Telegram и итерация по последним 50 сообщениям в каждом канале. Алгоритм анализирует содержимое каждого сообщения и проверяет, содержатся ли в нём упоминания акций. Если найдено совпадение, к соответствующему тикеру добавляется количество просмотров сообщения;

– получение статистики просмотров. Для более точной аналитики парсер использует сайт Telemetr, откуда с помощью библиотеки BeautifulSoup извлекается среднее количество просмотров на пост в каждом из каналов. Эти данные служат в качестве коэффициента веса упоминания - чем больше просмотров, тем большее влияние оказывает это упоминание в аналитике [8];

– фиксация новых сообщений. Если парсер обрабатывает новое сообщение, он сохраняет ID последнего просмотренного сообщения в backend, чтобы не анализировать одни и те же посты повторно;

– подсчёт упоминаний и обновление счётчиков. Сервис ведёт подсчёт упоминаний каждой акции за текущую сессию и отправляет эти данные на сервер. Backend либо обновляет существующий счётчик упоминаний, либо создаёт новый, если данные поступили впервые за день;

– периодический запуск. Парсинг запускается в цикле каждые 30 минут, обеспечивая регулярное обновление аналитики и своевременное реагирование на рыночные обсуждения в Telegram.

Этот микросервис выполняет важную аналитическую задачу - фиксирует активность по акциям в популярных Telegram-каналах и оценивает значимость упоминаний на основе просмотров. Эти данные можно использовать в системе как дополнительный индикатор интереса инвесторов и трейдеров, что особенно ценно для оперативного реагирования на рыночные события.

После детального разбора микросервиса telegram_parser, который выполняет функции сбора и предварительной обработки данных из телеграм-каналов, можно перейти к следующему важному компоненту системы – микросервису telegram_bot.

Если parser отвечает за "невидимую" часть работы с Telegram – автоматический парсинг и передачу данных в backend, то telegram_bot взаимодействует напрямую с конечными пользователями через интерфейс Telegram. Это – пользовательское представление всей системы внутри мессенджера. Бот служит связующим звеном между пользователем и

функциональностью платформы, предоставляя доступ к основным возможностям через команды, кнопки и чат-интерфейс.

Микросервис telegram_bot представляет собой интеграционный компонент системы, реализующий полноценного Telegram-бота на основе библиотеки Aiogram. Его основное назначение – предоставить конечным пользователям интерфейс для взаимодействия с платформой внутри мессенджера Telegram. Этот микросервис фактически служит пользовательским фронтендом, обеспечивая доступ к ключевым функциям системы, таким как авторизация, привязка аккаунтов, подтверждение сессий и возможное расширение сценариев взаимодействия в будущем [1].

С самого начала своей работы бот загружает необходимые параметры из конфигурационного файла .env, в числе которых находятся токен для Telegram API, адрес backend-сервиса, а также идентификатор и пароль сервисного аккаунта, с помощью которых осуществляется внутренняя авторизация микросервиса в системе. После старта микросервис инициирует запрос к backend для получения авторизационных cookie, которые будут использоваться в последующих вызовах защищённых маршрутов. Это позволяет обеспечить аутентификацию при взаимодействии с другими микросервисами, в частности с backend-частью.

Одной из ключевых функций telegram-бота является реализация FSM-машины (finite state machine), которая обрабатывает процесс логина пользователя через Telegram. В этом сценарии пользователь инициирует авторизацию с помощью команды, после чего бот последовательно запрашивает логин и пароль. Полученные данные передаются на backend, где производится проверка учётных данных и при успешной валидации Telegram-аккаунт пользователя привязывается к его записи в базе данных. Этот процесс реализован через систему состояний и позволяет осуществлять безопасную и контролируемую авторизацию пользователей прямо в мессенджере.

Помимо авторизации, бот выполняет обработку callback-запросов, поступающих от инлайн-кнопок. Например, предусмотрен специальный

механизм для подтверждения действий, таких как верификация активных сессий пользователей. Когда администратор или иной уполномоченный пользователь получает уведомление с инлайн-кнопкой подтверждения, бот извлекает идентификатор сессии и флаг одобрения, затем инициирует запрос к соответствующему маршруту backend. После обработки запроса пользователю отображается сообщение с итогом операции, отражающим статус подтверждения.

Процесс обработки сообщений и событий построен на системе диспетчеризации событий Telegram. Для этого используется объект Dispatcher, который отслеживает команды, состояния, callback-запросы и обеспечивает маршрутизацию логики в соответствующие функции-обработчики. Запуск polling-процесса обеспечивает непрерывное подключение к Telegram API и позволяет получать обновления в реальном времени, тем самым реализуя полноценную реактивную систему общения.

Таким образом, микросервис telegram_bot является важным связующим элементом архитектуры, обеспечивая прямой канал взаимодействия пользователей с системой. Он отвечает за управление сессиями, авторизацию через Telegram и подтверждение действий, и в перспективе может быть расширен для реализации дополнительных пользовательских функций, таких как получение уведомлений, управление подписками или взаимодействие с контентом платформы. Этот микросервис завершает цикл интеграции системы с Telegram, обеспечивая полное покрытие как внутренней логики (парсинг, хранение, управление), так и пользовательского интерфейса.

После детального рассмотрения микросервиса telegram_bot, который обеспечивает взаимодействие пользователей с системой через интерфейс мессенджера Telegram, логично перейти к следующему ключевому элементу архитектуры – микросервису manager. В отличие от предыдущего компонента, ориентированного на внешнее взаимодействие, manager выполняет функции координации, планирования и автоматического запуска внутренних процессов. Именно он выступает связующим и управляющим звеном между всеми

остальными частями системы, реализуя логику задач по расписанию, инициируя обновления, проверки и другие системные действия без участия пользователя.

Микросервис manager выполняет роль связующего и управляющего компонента, который координиет обмен сообщениями между внешними источниками данных (например, брокерскими API) и внутренней инфраструктурой системы через очередь RabbitMQ.

RabbitMQ обеспечивает асинхронное взаимодействие между различными компонентами системы. RabbitMQ используется здесь в качестве брокера сообщений – надёжного промежуточного звена между отправителями (producer) и получателями (consumer), позволяя компонентам системы обмениваться данными без необходимости прямой связи и синхронного ожидания ответа.

В контексте данного микросервиса RabbitMQ решает сразу несколько важных задач:

– обеспечивает очередь задач для отправки и получения данных от внешних источников. Например, микросервис manager формирует сообщения (в виде JSON-структур) с запросами на получение исторических свечей, обновление текущих цен, объёмов торгов или данных по стакану заявок, и публикует их в очередь. Далее другие специализированные микросервисы (например, взаимодействующие с брокерским API) подписаны на эту очередь и обрабатывают поступающие задачи. Такой подход полностью устраняет необходимость прямого запроса и ожидания ответа от сторонних сервисов – каждый компонент работает независимо, а RabbitMQ гарантирует доставку сообщений;

– выступает как буфер и средство масштабирования. Он позволяет эффективно распределять нагрузку между множеством потребителей сообщений. Если в будущем нагрузка возрастёт – можно будет просто добавить больше микросервисов-обработчиков, которые будут параллельно забирать задачи из очереди. Это делает систему масштабируемой и устойчивой к перегрузкам;

– повышает надёжность системы и устойчивость к сбоям. Даже если один из компонентов временно недоступен, сообщения в очереди сохраняются и не теряются. После восстановления компонента он может продолжить обработку задач с того места, на котором остановился.

Микросервис manager использует отдельные очереди для разных типов сообщений (например, для свечей, стаканов, списков акций), а также проверяет их состояние через HTTP-интерфейс RabbitMQ Management API. Это позволяет реализовать умную логику управления загрузкой – если в очередях нет задач, менеджер инициирует новые загрузки, тем самым поддерживая актуальность данных. В результате RabbitMQ в этой системе выполняет функцию центральной коммуникационной шины, разделяющей источники задач и их исполнителей. Он обеспечивает независимость компонентов, упрощает их масштабирование, снижает связанность, улучшает обработку ошибок и даёт гибкость при развитии архитектуры. Благодаря его использованию микросервис manager работает надёжно, масштабируемо и эффективно управляет циклом данных в реальном времени.

manager регулярно отправляет запросы на обновление торговых данных, инициирует первичную и периодическую загрузку информации об инструментах и свечах, а также отправляет уведомления пользователям Telegram при возникновении определённых условий. Он также обрабатывает входящие сессии пользователей, требующих ручного подтверждения, и отправляет им соответствующие интерактивные сообщения через Telegram-бота.

Настройка микросервиса начинается с файла config.py, в котором происходит загрузка параметров подключения к RabbitMQ из переменных окружения (.env-файла). Далее определяется функция get_connection, создающая и возвращающая соединение с сервером RabbitMQ с использованием переданных параметров, включая хост, порт, логин, пароль и другие тайм-ауты. Также присутствует функция configure_logging, отвечающая за настройку логгирования по переданному шаблону формата и уровню логирования (по умолчанию – INFO).

Файл publisher.py содержит основной класс Publisher, реализующий всю логику работы микросервиса. В конструкторе класса происходит инициализация соединения с RabbitMQ, создание очередей для отправки и получения сообщений, а также авторизация в микросервисе backend с получением session cookies. Далее создаётся экземпляр Telegram-бота (через aiogram), который используется для взаимодействия с пользователями при подтверждении новых сессий или отправке уведомлений.

Внутри класса определён вспомогательный класс RMQData, представляющий структуру для сериализации сообщений, отправляемых в очередь. Метод serialize превращает объект в строку, готовую для передачи через RabbitMQ. Метод get_message_count позволяет получить количество сообщений в указанной очереди, обращаясь к HTTP-интерфейсу RabbitMQ Management API.

Основной метод start запускает бесконечный цикл, в котором каждую секунду выполняются определённые проверки. Каждые 10 секунд осуществляется проверка количества сообщений в очередях запросов и ответов. Если в обеих очередях отсутствуют сообщения, запускаются методы update_prices_and_volumes и update_orders_book, инициирующие запросы на обновление данных о свечах и стакане заявок. Также в этом цикле раз в 10 секунд вызывается метод process_unapproved_sessions, отправляющий Telegram-сообщения пользователям, ожидающим подтверждения входа. Раз в минуту срабатывает метод check_volumes_notifications, который проверяет, не превысили ли объёмы торгов пользовательские пороги, и если да – отправляет соответствующее уведомление пользователю через Telegram.

Функция update_prices_and_volumes формирует запросы на получение свечей (candles) для всех активных акций. Она сначала запрашивает список всех активных тикеров из backend, затем для каждого тикера проверяет, есть ли у него уже загруженные объёмы. Если объёмы отсутствуют, то инициируется полная историческая загрузка данных. Если объёмы уже есть – запрашиваются только последние данные за указанный период.

Функция update_orders_book работает аналогично, но направлена на обновление данных по стаканам заявок. Для каждого активного тикера отправляется запрос на получение книги заявок с глубиной 50 уровней.

Функция process_unapproved_sessions загружает все непроверенные сессии пользователей и отправляет соответствующим Telegram-пользователям сообщение с двумя кнопками – для подтверждения или отклонения попытки входа. Эти кнопки реализуются с помощью InlineKeyboard и callback-событий.

Функция check_volumes_notifications выполняет проверку, были ли по какому-либо активу зафиксированы объёмы торгов, превышающие пользовательские пороги. В случае обнаружения таких событий, пользователю через Telegram отправляется соответствующее сообщение.

Файл main.py служит точкой входа. В нём происходит импорт всех необходимых компонентов, настройка логгирования, инициализация подключения к RabbitMQ и создание экземпляра класса Publisher. Далее запускается основной цикл работы микросервиса через метод start.

Таким образом, микросервис manager выполняет роль автоматического диспетчера и координационного звена. Он непрерывно следит за состоянием очередей, инициирует обновление рыночных данных, отслеживает активности пользователей и обеспечивает их информирование. Через взаимодействие с Telegram и RabbitMQ он связывает в единое целое внутреннюю аналитику и внешнюю инфраструктуру.

После описания микросервиса manager, который отвечает за генерацию задач, их публикацию в очереди и контроль за обновляемостью рыночных данных, следующим логичным звеном в цепочке обработки становится микросервис handler. Если manager играет роль координатора и постановщика задач, то handler – это исполнитель, который эти задачи принимает, обрабатывает и обновляет базу данных на основе полученной информации. Именно он взаимодействует с брокерскими API, извлекает актуальные данные и сохраняет их в систему. Теперь рассмотрим этот микросервис подробнее.

Микросервис handler отвечает за обработку и анализ сообщений, поступающих из очередей RabbitMQ. Его главная задача – принять входные данные, полученные от других компонентов системы (например, от микросервиса manager), и, при необходимости, выполнить дополнительные действия, такие как запрос к backend'у, логгирование событий или генерация ответов. Архитектурно он представляет собой единичный потребитель сообщений, работающий в рамках одного потока.

Основной точкой входа является файл main.py. Внутри него происходит первичная настройка среды: загружается .env-файл, настраивается логгирование, инициализируются переменные окружения, такие как адрес backend-сервиса, имя текущего сервиса и его пароль, формат даты и имя очереди, из которой будут поступать сообщения. Затем, с помощью функции get_connection из модуля config.py, устанавливается подключение к RabbitMQ. После этого создаётся экземпляр класса Handler, которому передаётся открытый канал RabbitMQ и остальные параметры, необходимые для работы. Объект handler запускается через метод start, который, как можно предположить, блокирует поток до тех пор, пока обрабатываются сообщения, – реализуя бесконечный цикл обработки.

Файл config.py содержит утилиты, необходимые для подключения к брокеру сообщений RabbitMQ. Через переменные окружения загружаются параметры подключения: хост, порт, логин и пароль. Создаётся соединение с помощью библиотеки pika, которая обеспечивает взаимодействие Python-приложения с RabbitMQ. Конфигурация соединения включает в себя отключение heartbeat (что полезно при долгих операциях) и увеличенное значение blocked_connection_timeout, что позволяет системе устойчиво работать даже при длительных блокировках. Кроме подключения, config.py предоставляет метод configure_logging, настраивающий глобальное логгирование по заранее заданному шаблону.

Основная логика микросервиса реализована в файле handler.py. Этот файл содержит класс Handler, который берёт на себя задачу подписки на очередь и обработки поступающих сообщений. При создании экземпляра класса ему

передаются параметры подключения к RabbitMQ, параметры авторизации к backend и другие настройки. Сам метод start – это запуск основного цикла обработки, где каждый входящий элемент из очереди должен быть принят, разобран и соответствующим образом обработан. В зависимости от архитектуры, handler может выполнять вызовы к API, проводить вычисления, валидировать данные или направлять их дальше по системе.

Ключевым компонентом микросервиса является RabbitMQ – брокер сообщений, обеспечивающий асинхронное взаимодействие между микросервисами. В данном контексте RabbitMQ играет роль "связующего канала", через который другие сервисы, такие как manager, могут отправлять события или команды, которые затем подхватываются handler-ом. Это позволяет реализовать слабую связанность между компонентами системы и добиться высокой гибкости в масштабировании. Использование очередей особенно важно при работе с большими объёмами данных или событий, поскольку оно обеспечивает буферизацию и контроль нагрузки: handler может обрабатывать сообщения последовательно, в своём ритме, независимо от того, с какой скоростью они поступают от источника.

Таким образом, микросервис handler – это обработчик событий из очередей RabbitMQ, который в связке с backend и другими компонентами реализует асинхронную, расширяемую и отказоустойчивую архитектуру. Он принимает на себя нагрузку по промежуточной логике между генерацией событий и их финальной реализацией, позволяя гибко масштабировать систему.

После детального рассмотрения микросервиса handler, который выполняет обработку событий из очередей RabbitMQ и служит связующим звеном между генерацией данных и их последующей логической реализацией, следующим элементом в архитектуре становится микросервис api_worker. Этот компонент системы также играет важную роль в распределённой инфраструктуре, но с иной специализацией: он отвечает за выполнение целевых API-запросов – например, к внешним финансовым платформам, новостным агрегаторам или другим источникам, требующим сетевого взаимодействия. Микросервис api_worker

становится своего рода исполнительным агентом, получающим команды через очередь, выполняющим обращения к нужным сервисам и возвращающим результат в общий поток обработки. Далее подробно разберём его устройство, ключевые модули и принципы работы.

Микросервис api_worker обслуживает очереди RabbitMQ и взаимодействующего с внешним API брокера Tinkoff для получения рыночной информации. Он задуман как высоконагруженный компонент, предназначенный для обработки API-запросов в изолированном и асинхронном режиме. Основной задачей микросервиса является приём запросов из очереди requests_q, выполнение указанных в этих запросах методов через официальную библиотеку tinkoff.invest, сериализация результатов и отправка ответов обратно в очередь responses_q. Архитектура построена с фокусом на производительность и надёжность при работе с внешними API [10].

Файл config.py отвечает за конфигурацию соединения с RabbitMQ и настройку логирования. Здесь считываются параметры окружения, такие как хост, порт, имя пользователя и пароль для подключения к брокеру сообщений. Далее с использованием этих параметров создаётся соединение pika.BlockingConnection, на базе которого формируются каналы связи. Конфигурация логирования обеспечивает единый формат журналирования, что упрощает мониторинг и отладку при эксплуатации.

Файл main.py служит точкой входа в микросервис. В самом начале он загружает переменные окружения из файла .env, включая названия очередей, токен доступа к API Tinkoff, а также формат даты. После этого производится настройка логирования и инициируется соединение с RabbitMQ. Затем создаётся объект Worker – основной рабочий компонент сервиса, которому передаётся всё необходимое: каналы связи с RabbitMQ, очереди запросов и ответов, токен, логгер и формат даты. Вызов метода start() запускает бесконечный цикл потребления сообщений из очереди и их обработки.

Наиболее значимая логика находится в файле worker.py. Здесь определён вспомогательный класс RMQData, инкапсулирующий логику сериализации и

десериализации сообщений из очереди. Он позволяет извлекать из запроса метод вызова, аргументы и приоритет, а затем упаковывать обратно результат с добавленным полем payload. Далее следует основной класс Worker, который при инициализации создаёт соединение с Tinkoff API с помощью токена, а также регистрирует очереди RabbitMQ с поддержкой приоритета.

Метод start запускает бесконечное прослушивание очереди запросов и обрабатывает каждое новое сообщение через метод process_new_message. В процессе обработки сообщение сначала парсится, затем на основе заданного метода происходит вызов соответствующей функции API Tinkoff (например, instruments.shares, market_data.get_candles, market_data.get_order_book). В случае, если метод неизвестен или произошла ошибка при вызове API, сообщение логируется, но не останавливает работу сервиса. После успешного получения ответа он сериализуется и отправляется обратно в очередь ответов с сохранением приоритета.

RabbitMQ в данном контексте выступает в роли связующего звена между компонентами микросервисной архитектуры. Он позволяет эффективно распределять задачи по обработке API-запросов, обеспечивая надёжную передачу сообщений с возможностью задания приоритетов. Использование очередей даёт возможность обрабатывать большое количество запросов параллельно, масштабировать api_worker по горизонтали и отслеживать неуспешные операции без потерь данных. Кроме того, RabbitMQ обеспечивает устойчивость при временных сбоях сети или недоступности компонентов, так как все сообщения остаются в очереди до успешной обработки.

Таким образом, микросервис api_worker – это специализированный исполнитель запросов к Tinkoff API, организованный по асинхронной модели с применением очередей RabbitMQ. Он изолирует логику работы с внешним API от других компонентов системы, повышая надёжность, масштабируемость и отказоустойчивость архитектуры в целом.

Микросервисная архитектура приложения выстроена как комплексная и гибко масштабируемая система, в которой каждый компонент отвечает за строго

определённую зону ответственности, обеспечивая при этом высокий уровень модульности, устойчивости и отказоустойчивости. Архитектура разделена на отдельные микросервисы: backend, telegram_parser, telegram_bot, manager, handler, и api_worker, каждый из которых взаимодействует с другими через стандартизированные интерфейсы – будь то REST API, базы данных или брокер сообщений RabbitMQ.

Центральным звеном служит backend, который обеспечивает REST-интерфейс для frontend-клиента, обрабатывает пользовательские запросы, управляет сессиями, настройками, аналитикой и хранением данных. Этот микросервис реализован по модульной схеме и разделён на три группы маршрутов: user, admin и service, что упрощает поддержку и разграничение прав доступа.

Микросервисы telegram_parser и telegram_bot обеспечивают интеграцию с Telegram: первый собирает и обрабатывает упоминания акций в каналах, второй – служит интерактивным интерфейсом для подтверждения авторизации и привязки аккаунтов. Эти сервисы работают автономно, но плотно связаны с backend через API-запросы.

manager и handler обеспечивают очередь задач, обработку сообщений и передачу команд между компонентами через RabbitMQ. Это позволяет выстраивать асинхронные, не блокирующие цепочки вызовов, разгружать систему и обеспечивать устойчивость при высоких нагрузках.

Наконец, api_worker выполняет внешние API-запросы к брокеру (Tinkoff), обрабатывая их по приоритету и возвращая ответы в очереди. Такой подход позволяет вынести трудоёмкие операции в изолированный процесс, минимизируя влияние возможных задержек на другие части системы.

В целом, архитектура построена по принципам высокой декомпозиции, независимости компонентов и горизонтальной масштабируемости. Использование RabbitMQ как шины данных между микросервисами обеспечивает асинхронность, устойчивость и гибкость в распределении задач, а

чёткое разграничение логики позволяет легко сопровождать и расширять систему в будущем.

3.3. Frontend логика

Структура интерфейса разделена на несколько ключевых блоков. Левая панель содержит список отслеживаемых тикеров и настройки, позволяя пользователю быстро переключаться между разными инструментами. Центральная область отображает график стакана с актуальными данными по заявкам на покупку и продажу, что даёт возможность анализировать рыночные изменения в реальном времени. Справа расположены виджеты с дополнительной информацией: график новостей, график объёмов и индикаторы EMA50 и EMA200. Такая организация интерфейса позволяет пользователю оперативно получать всю необходимую информацию, не переключаясь между разными экранами.

Дизайн приложения выполнен в минималистичном стиле, что исключает перегруженность интерфейса и позволяет сосредоточить внимание на данных. Элементы тщательно расположены таким образом, чтобы пользователь мог легко ориентироваться в информации и быстро находить нужные данные. Дизайн также адаптивен, что обеспечивает корректное отображение интерфейса на разных устройствах.

Важно отметить, что разработка дизайна осуществлялась в инструменте Figma – это современное средство для проектирования пользовательских интерфейсов, которое позволяет эффективно организовывать рабочий процесс, в том числе совместную работу, и вносить правки в макеты в реальном времени. В процессе проектирования были продуманы как основные элементы интерфейса, так и их поведение при взаимодействии, что в дальнейшем значительно упростило реализацию на фронтенде [39].

Интерактивные элементы, такие как кнопки, фильтры и настраиваемые графики, позволяют пользователю персонализировать отображение информации

в соответствии со своими предпочтениями. Таким образом, дизайн приложения сочетает в себе функциональность, удобство использования и эстетическую привлекательность, делая его эффективным инструментом для мониторинга фондового рынка.

На рисунке 5 представлен макет пользовательского интерфейса, разработанный в Figma [32].



Рисунок 5 – Интерфейс приложения

После завершения этапа проектирования интерфейса в Figma началась реализация клиентской части приложения. На основе визуального макета был создан полноценный пользовательский интерфейс, в котором каждый элемент получил не только визуальное, но и функциональное воплощение. Разработка велась с ориентацией на адаптивность, отзывчивость и высокую интерактивность, что позволило добиться удобного взаимодействия пользователя с системой независимо от типа устройства.

Одним из первых реализованных компонентов стала страница авторизации. Она автоматически отображается, если у пользователя отсутствует cookie-файл с действительной сессией, что обеспечивает базовую защиту и контроль доступа. Страница включает два текстовых поля для ввода логина и пароля, а также кнопку "Войти". После нажатия происходит отправка данных на

сервер, где осуществляется проверка подлинности. В случае двухфакторной авторизации, пользователю дополнительно необходимо подтвердить вход через Telegram-бот. Только после успешного подтверждения пользователь получает cookie, сессия сохраняется, и ему становится доступен основной функционал приложения. Также, сразу после авторизации, клиент отправляет серию запросов к серверу для загрузки необходимых данных: тикеров, графиков, настроек, индикаторов и пользовательских параметров.

После входа пользователь попадает на главный экран приложения, где реализована левая панель навигации - важный элемент интерфейса, обеспечивающий доступ ко всем основным функциям. Верхний блок панели содержит кнопку «Настройки», открывающую модальное окно. При её активации затемняется фон основного содержимого, а по центру появляется всплывающее окно с возможностью прокрутки. В модальном окне отображается список пользовательских параметров, где каждая настройка сопровождается пояснением. Подсказка с полным описанием появляется при наведении на текстовое поле. Пользователь может изменить значение и сохранить его. После этого автоматически отправляется PUT-запрос на сервер с обновлёнными параметрами. Далее происходит повторная загрузка всех графиков и аналитических данных в соответствии с обновлёнными настройками.

Под блоком настроек расположен фильтр выбора инструмента. При наведении курсора появляется выпадающее меню с перечнем типов активов - например, акции, облигации, ETF и прочее. Выбор определённого инструмента инициирует запрос к серверу, который возвращает актуальный список тикеров по выбранному фильтру.

Ниже расположен поисковый блок. Это интерактивное текстовое поле с мгновенной фильтрацией списка акций. Поиск осуществляется как по короткому тикеру (например, "T"), так и по полному названию компании. Результаты отображаются динамически по мере ввода. Каждый элемент списка представляет собой карточку с краткой информацией об активе и иконкой добавления («+»). Нажатие на иконку генерирует POST-запрос на сервер, добавляющий тикер в

пользовательский список наблюдения. Сервер сохраняет эту настройку и возвращает обновлённый список активов. Эти данные также кешируются в локальном хранилище для повышения производительности.

Для повышения удобства пользователю также доступна возможность сортировки списка активов - по алфавиту, активности, дате добавления или другим параметрам. При выборе одного из вариантов сортировки отправляется соответствующий запрос на сервер или фильтрация выполняется локально, если данные уже были загружены.

Дополнительно стоит отметить, что в левой панели предусмотрена система индикации статуса актива - например, изменение цены, объёма или упоминаний в Telegram. Эти индикаторы обновляются в реальном времени через WebSocket-соединение, позволяя отслеживать изменения без необходимости ручного обновления страницы.

Также планируется реализация функционала группировки активов по папкам или меткам, чтобы пользователь мог структурировать свой список наблюдения. Это особенно актуально при большом количестве отслеживаемых инструментов.

Таким образом, левая панель представляет собой не просто навигационную колонку, а мощный инструмент управления настройками, фильтрацией и добавлением активов в индивидуальный профиль пользователя. Она служит связующим звеном между интерфейсом и логикой приложения, обеспечивая высокий уровень кастомизации и интерактивности.

Для реализации всех графических компонентов пользовательского интерфейса была выбрана библиотека amCharts. Это отличный инструмент визуализации данных, предназначенный для построения интерактивных и настраиваемых графиков на JavaScript. Библиотека предоставляет обширный набор диаграмм, включая линейные, столбчатые, свечные графики, а также специализированные компоненты вроде тепловых карт, гистограмм распределения и прочих. Среди основных преимуществ amCharts - высокая производительность, простота кастомизации, поддержка темы (в нашем случае -

тёмная), а также наличие собственного адаптивного механизма, который автоматически перестраивает графики под разные разрешения экранов [68].

Одним из ключевых визуальных элементов интерфейса является график стакана, отображаемый в центральной части экрана при выборе конкретного тикера. График стакана позволяет анализировать распределение крупных ордеров на покупку и продажу в зависимости от цены, а также оценивать активность участников рынка в различных диапазонах цен.

Структурно стакан включает несколько элементов:

– текущая цена актива, отображаемая в верхней части блока. Это актуальная рыночная цена акции, которая обновляется в реальном времени, независимо от выбранного временного интервала;

– список крупных заявок - две колонки, где слева отображаются заявки на покупку, а справа - на продажу. Каждая строка содержит объём и цену. Этот список формируется на основе агрегированных данных по крупным ордерам за заданный период, выбранный в дате;

– гистограмма «Суммы» - визуализирует объемы заявок на каждом ценовом уровне. Левый столбец (бирюзового оттенка) отражает объёмы на покупку, а правый (сиреневый) - на продажу. Данные отображаются за выбранный временной промежуток, позволяя пользователю исследовать рыночную активность в динамике;

– блок уровней ещё один важный компонент стакана, представляющий собой агрегированную аналитику. Здесь рассчитываются уровни с наибольшей плотностью заявок за выбранный период. Помимо объёма, указывается процент пробития уровня: это метрика, отражающая, в скольких процентах дней (в рамках интервала) цена действительно пробивала указанный уровень. Например, если уровень был пробит в 92% случаев - это сигнал о его высокой значимости и слабом сопротивлении, тогда как 100% пробития говорит о надёжной зоне спроса/предложения. Такой подход помогает оценить историческую реакцию цены на ключевые уровни.

Рядом с графиком стакана предусмотрена кнопка с иконкой в виде мусорного ведра - это механизм удаления тикера из пользовательского списка наблюдения. При её нажатии отправляется DELETE-запрос на сервер с идентификатором тикера, после чего:

– тикер удаляется из локального списка;

– пользователь может вручную снова добавить тикер через фильтр или поиск, если захочет продолжить наблюдение.

Выбор временного интервала осуществляется через дату в правом верхнем углу блока стакана. Пользователь может задать любую дату «От» и «До», и в зависимости от интервала будет происходить перерасчёт:

– объёмов заявок на каждом уровне;

– значений уровней и их статистики пробития;

– отображаемых гистограмм.

Таким образом, стакан в приложении - это не просто отображение текущих ордеров, а мощный аналитический инструмент, который позволяет глубоко оценивать рыночную активность по каждой акции за произвольный период, находить зоны скопления интереса и принимать решения на основе исторической реакции рынка.

График «упоминания в TG» Этот график представляет собой временную столбчатую диаграмму, отображающую, сколько раз выбранный тикер был упомянут в Telegram-каналах за каждый день в пределах указанного периода. Он построен на основе собранных данных из публичных и специализированных каналов, которые бот приложения мониторит в режиме реального времени.

Основные функции и особенности:

– ось X - временная шкала (по умолчанию - дни, но пользователь может сменить интервал на недели или месяцы);

– ось Y - количество уникальных упоминаний;

– каждый столбик графика - это сумма всех упоминаний за конкретный день. Высокие пики указывают на рост интереса к бумаге.

– цветовая палитра может изменяться в зависимости от контекста.

– при наведении отображается данные с точной датой и количеством упоминаний;

– справа от заголовка есть селектор периода и кнопки управления кнопки плюс и минус - для масштабирования временного диапазона.

Этот график особенно полезен для оценки социального фона и информационного шума, который может предшествовать или сопровождать резкие ценовые движения.

График объёмов - один из ключевых аналитических инструментов, предоставляющий пользователю информацию о биржевой активности по каждой бумаге. Он также выполнен в формате столбчатой диаграммы, но в отличие от графика упоминаний, показывает торговые объёмы в деньгах или лотах.

Что отображает:

– ось X - временная шкала, аналогично предыдущему графику;

– ось Y - объёмы торгов;

– каждый столбец показывает торговый оборот за конкретный день;

– горизонтальная пунктирная линия - это средний объём за период. Она служит ориентиром, чтобы сразу видеть дни, когда объёмы значительно превышали средние значения;

– цвета столбиков варьируются: можно настраивать отображение по направлению дня (например, зелёный - рост, красный - падение цены);

– позволяет выявить дни с повышенным интересом к активу, что может свидетельствовать о важных новостях или выходе отчётности;

– высокий объём на фоне роста или падения цены - один из главных сигналов для трейдеров и инвесторов;

– можно быстро визуально сопоставить с графиком Telegram-упоминаний для выявления корреляции между информационным шумом и рыночной реакцией.

Этот компонент визуализирует две экспоненциальные скользящие средние - EMA200 и EMA50, которые являются популярными техническими индикаторами в трейдинге:

– EMA50 - отображает среднюю цену акции за последние 50 торговых дней, придавая больший вес последним значениям;

– EMA200 - более «долгосрочная» средняя, рассчитываемая по 200 дням;

– обе линии накладываются на ценовой график или идут параллельно как отдельный слой.

– График упоминаний и объёмов работает в синхронизации с остальными компонентами интерфейса:

– при изменении тикера, обновляются все графики в соответствии с новым идентификатором инструмента;

– при изменении временного диапазона (вверху на панели стакана) все графики перестраиваются, подгружая данные только за указанный период;

– все визуальные компоненты оптимизированы для быстрого рендеринга и работают даже при большом объёме данных (благодаря оптимизациям amCharts и ленивой загрузке данных с сервера).

Интерфейс получился функциональным, современным и глубоко аналитическим - он объединяет визуализацию ключевых рыночных метрик (объёмы, уровни, EMA, соцсигналы) в едином интерактивном пространстве. Благодаря использованию amCharts, все графики выглядят стильно, интерактивно и масштабируемо.

Панель стакана позволяет не просто следить за текущей ценой, но и анализировать поведенческие паттерны трейдеров на основе крупных заявок и исторических уровней.

Информационные графики справа дают понимание - почему происходит движение, подкрепляя технический анализ социальным и новостным контекстом.

Всё это делает frontend не просто витриной данных, а инструментом принятия решений, сочетающим глубину анализа с интуитивным UX.

3.4. Реализация виртуального хостинга

Для того чтобы обеспечить корректную и стабильную работу приложения на удалённом сервере, каждый микросервис архитектуры был обёрнут в изолированное Docker-среду с использованием собственного Dockerfile. Это решение позволяет упростить процесс сборки, конфигурации и развертывания, а также свести к минимуму различия между средами разработки и эксплуатации. Все Dockerfile были разработаны по общему шаблону, но адаптированы под специфику каждого сервиса. Ниже приводится подробное описание каждого из них.

Микросервис backend обслуживает всю бизнес-логику приложения и взаимодействует с базой данных и фронтендом. В его Dockerfile в качестве базового образа используется python:3.10, что обеспечивает совместимость с используемыми библиотеками. В рабочую директорию /backend копируется файл зависимостей requirements.txt, и далее происходит установка всех библиотек через pip. После этого весь исходный код проекта копируется внутрь контейнера. Дополнительно задаются переменные окружения LANG, LC_ALL и PYTHONUNBUFFERED, что обеспечивает корректную работу с кодировками и отключение буферизации вывода. В качестве точки входа указан файл main.py, который запускается при старте контейнера.

Микросервис api_worker работает с внешним API (в частности, Tinkoff Invest) и обрабатывает команды, приходящие через RabbitMQ. Его Dockerfile полностью повторяет структуру backend, но с рабочей директорией /api_worker, что обеспечивает изоляцию окружения и независимость при сборке и запуске.

Микросервис telegram_parser отвечает за парсинг сообщений из Telegram-каналов. Dockerfile идентичен предыдущим: используется Python 3.10, копируются зависимости и исходный код, а запуск осуществляется через main.py из директории /telegram_parser.

Микросервис telegram_bot реализует Telegram-бота, который взаимодействует с пользователем, позволяет привязывать Telegram-аккаунты и

подтверждать сессии. Его контейнерная сборка также основана на Python 3.10, с теми же стандартами конфигурации. Рабочая директория – /telegram_bot.

Микросервис manager выступает как центральный управляющий узел, получающий команды через RabbitMQ, перенаправляющий их соответствующим воркерам и обеспечивающий логику очередей и обработки приоритетов. В Dockerfile используется такой же подход: установка зависимостей, копирование кода, настройка окружения и запуск из main.py.

Микросервис handler обрабатывает ответы от api_worker и формирует агрегированные данные, необходимые для аналитики. Его Dockerfile ничем не отличается по структуре, меняется только рабочая директория – /handler.

Микросервис frontend реализует клиентскую часть приложения, написанную на JavaScript с использованием Node.js. В Dockerfile в качестве базового образа выбран node:18-alpine – облегчённая версия Node.js, обеспечивающая быструю сборку. Сначала в контейнер копируется package.json, после чего выполняется установка зависимостей через npm install. Затем внутрь контейнера переносится весь проект, и запускается команда npm run dev, предназначенная для старта сервера в режиме разработки.

Таким образом, каждый Dockerfile был создан с учётом потребностей конкретного микросервиса, но придерживается общего шаблона: минимальная базовая система, установка зависимостей, копирование исходного кода, задание рабочих параметров среды и определение команды запуска. Благодаря этому архитектура получилась унифицированной, масштабируемой и готовой к работе в различных окружениях. В следующем разделе будет рассмотрен файл docker-compose.yml, который связывает все микросервисы воедино и обеспечивает их совместную работу.

После разработки и контейнеризации всех микросервисов, следующим ключевым этапом стала их интеграция и координация в рамках общего окружения. Для этого был создан файл docker-compose.yaml, который объединяет все сервисы проекта, включая базы данных, брокер сообщений и отдельные микросервисы, в единую систему. Использование docker-compose

позволило не только автоматизировать развёртывание приложения, но и обеспечить его стабильную и согласованную работу вне зависимости от окружения. Далее представлено подробное описание структуры файла и назначения каждого раздела.

В корне конфигурации указывается версия схемы (version: "3"), после чего в секции services описываются все используемые контейнеры. Первым идёт сервис frontend, реализующий клиентскую часть приложения. Он собирается из локальной директории frontend и пробрасывает порт 3000, чтобы интерфейс был доступен извне. Далее идёт database – сервис PostgreSQL, отвечающий за хранение всей информации системы. В образе используется облегчённая версия PostgreSQL (postgres:17.2-alpine), указываются переменные окружения для создания базы, пользователя и пароля, а также настраивается монтирование данных с помощью тома db-data. Через healthcheck задаются параметры проверки доступности базы данных, чтобы избежать преждевременного запуска зависимых сервисов. Также добавлена настройка логирования с ограничением на объём логов.

Следующий важный элемент – rabbitmq, брокер сообщений, обеспечивающий асинхронную коммуникацию между сервисами. Он запускается с web-интерфейсом управления (rabbitmq:3-management-alpine), пробрасываются порты 5672 (основной) и 15672 (панель управления). Указываются переменные окружения с логином и паролем, а также healthcheck, определяющий готовность сервиса.

Микросервис api_worker зависит от rabbitmq и запускается только при его готовности. Он собирается из директории api_worker и использует .env файл с переменными окружения. Логика обработки сообщений с приоритетами и связи с брокером сообщений реализуется через RabbitMQ. Аналогично работает сервис backend, который зависит от базы данных. Он отвечает за REST API, авторизацию, работу с данными и пользовательские настройки. Здесь также пробрасывается порт 5000 для доступа к API извне.

Manager – это управляющий сервис, обрабатывающий запросы от различных компонентов и направляющий их в очередь. Он зависит от запуска backend и готовности rabbitmq, что гарантирует корректную последовательность инициализации. Сервис handler, обрабатывающий ответы и формирующий агрегации, также зависит от этих же двух компонентов.

Сервис telegram_bot реализует Telegram-бота и взаимодействует с пользователями. Он зависит только от backend, так как использует его API для верификации и привязки аккаунтов. В свою очередь, telegram_parser занимается сбором новостей из Telegram-каналов и тоже зависит от backend, так как записывает информацию в базу через API.

В завершении конфигурации определяется том db-data, обеспечивающий сохранность данных базы даже при перезапуске контейнеров. Все микросервисы используют общий .env файл, что позволяет централизованно управлять переменными окружения.

Таким образом, docker-compose.yaml стал ядром инфраструктурной логики приложения, связывая все микросервисы, обеспечивая правильную последовательность запуска, изоляцию окружений и централизованное управление. Благодаря этому вся система может быть развёрнута на удалённом сервере одной командой – docker-compose up --build -d, что существенно упрощает эксплуатацию, масштабирование и перенос приложения между серверами.

Выводы по третьей главе

В данной главе была выполнена комплексная реализация архитектуры программной системы, включающая разработку базы данных, проектирование микросервисной структуры, описание клиентской части и внедрение системы виртуального хостинга. Работа была организована поэтапно, что позволило детально проанализировать и выстроить устойчивую, масштабируемую и функционально полную архитектуру.

Первоначально была спроектирована база данных, играющая фундаментальную роль в работе всей системы. В рамках инфологического и физического проектирования были определены ключевые сущности: пользователи, сессии, акции, настройки, заявки, объёмы торгов, новости и прочие. Каждая таблица связана логически и структурно, обеспечивая целостность, производительность и согласованность данных. Были установлены связи между таблицами (один-к-одному, один-ко-многим, многие-ко-многим), а также заданы ключевые поля и типы данных, обеспечивающие оптимизацию хранения и выборки информации. Особое внимание было уделено описанию пользовательских настроек, новостных счётчиков и аналитических параметров, используемых в алгоритмах приложения.

Следующим важным этапом стало построение микросервисной архитектуры backend. Микросервисы были распределены по функциям: модуль backend обрабатывает API-запросы, авторизацию, хранение и агрегацию данных; сервисы manager, handler и api_worker отвечают за фоновые вычисления, маршрутизацию сообщений и взаимодействие с внешними API. Для каждого микросервиса была описана бизнес-логика, маршруты, реализуемые функции, зависимости и способы взаимодействия. Отдельно выделены роли сервисов telegram_parser и telegram_bot, реализующих интеграцию с Telegram: один собирает новостные данные, второй обеспечивает взаимодействие с пользователями, включая подтверждение сессий. Все модули организованы в виде независимых компонентов, объединённых через брокер сообщений RabbitMQ, что обеспечивает надёжность, масштабируемость и асинхронность.

Параллельно была рассмотрена структура frontend-интерфейса, реализованного на современном JavaScript-фреймворке. Он обеспечивает пользователю доступ к аналитике, настройкам, уведомлениям и интерактивным графикам. Через REST API frontend получает уже предобработанные данные и визуализирует их в доступной форме. Были реализованы интерфейсы работы с тикерами, настройками EMA, новостями, фильтрами и сигналами. Архитектура фронтенда ориентирована на модульность и удобство расширения.

Финальной частью главы стала реализация виртуального хостинга. Каждый микросервис был контейнеризирован с помощью Docker, а общее взаимодействие организовано с помощью docker-compose. Были созданы индивидуальные Dockerfile для всех сервисов, включая backend, frontend, Telegram-сервисы и API-обработчики. В docker-compose.yaml описана общая схема инфраструктуры, определены зависимости, параметры запуска, проброс портов и переменные окружения. Таким образом, вся система может быть легко развёрнута и масштабирована на любом удалённом сервере, что существенно упрощает эксплуатацию, обновление и мониторинг приложения.

В итоге, третья глава представляет собой всестороннюю реализацию прикладной архитектуры, охватывающую все ключевые аспекты современной микросервисной системы: от проектирования базы данных до автоматизированного развёртывания в изолированной среде. Такое решение обеспечивает надёжность, гибкость и готовность к дальнейшему развитию программного продукта.

ЗАКЛЮЧЕНИЕ

В ходе разработки и проектирования веб-приложения для анализа и визуализации данных о российском фондовом рынке была проделана обширная и многоуровневая работа, охватывающая как теоретические, так и практические аспекты разработки современного программного обеспечения. Вся работа была логически разделена на три взаимосвязанных главы, каждая из которых последовательно приближала нас к главной цели – созданию функционального, масштабируемого и надёжного веб-приложения, позволяющего пользователям анализировать рыночные данные и принимать инвестиционные решения на основе объективной информации.

В первой главе был проведён всесторонний анализ предметной области, в рамках которого рассмотрены особенности российского фондового рынка, его ключевые инструменты, участники, а также специфика публичной информации и сложности, связанные с её сбором, агрегацией и интерпретацией. Были обозначены основные потребности целевой аудитории, включающей как частных инвесторов, так и аналитиков, стремящихся к оперативному доступу к данным о котировках, объёмах торгов, новостях, уровнях поддержки и сопротивления, индикаторах технического анализа и другой рыночной информации. Особое внимание было уделено актуальности задачи – в условиях высокой волатильности рынков и геополитических изменений наличие персонализированного инструмента для анализа стало важной потребностью. Кроме того, были сформулированы ключевые требования к системе: быстродействие, гибкость, возможность масштабирования, интеграция с внешними источниками и наличие интуитивного пользовательского интерфейса.

Во второй главе был проведён обзор технологий и инструментов, которые легли в основу реализации проекта. Были рассмотрены языки программирования, библиотеки и фреймворки, актуальные для создания современных распределённых приложений. В качестве backend-решения был выбран Python и асинхронный веб-фреймворк FastAPI, обладающий высокой

производительностью и отличной поддержкой документации. Для обработки фоновых задач и распределённых вычислений были использованы брокер сообщений RabbitMQ и архитектурный подход микросервисов, обеспечивающий гибкость и устойчивость всей системы. В качестве клиентской части выбран современный JavaScript-фреймворк NextJS, позволяющий строить динамичные и отзывчивые интерфейсы с поддержкой визуализации данных. Отдельно были рассмотрены системы контейнеризации и оркестрации, в частности Docker и docker-compose, обеспечивающие удобство развёртывания, тестирования и масштабирования проекта. Также были проанализированы принципы работы с базами данных, выбор структуры таблиц, методов оптимизации запросов, кэширования и использования аналитических функций.

Третья глава стала основной и самой масштабной частью разработки приложения, в которой была реализована вся архитектура приложения. Разработка началась с проектирования базы данных – инфологическая и физическая модели были разработаны с учётом всех логических связей между сущностями: пользователями, акциями, сессиями, новостями, объёмами торгов, заявками и пользовательскими настройками. Особое внимание было уделено гибкой настройке аналитических параметров, позволяющих пользователю кастомизировать поведение системы под собственные цели.

Затем была реализована полноценная микросервисная архитектура. Backend был разделён на логические блоки: маршруты пользователя, администратора и сервисов. Каждый микросервис (например, manager, handler, api_worker) получил свою изолированную зону ответственности и взаимодействует с другими частями системы через очередь сообщений RabbitMQ, что обеспечивает отказоустойчивость и асинхронность обработки задач. Были описаны модули сбора данных, расчёта EMA, анализа новостного фона, крупных заявок, графиков и уровней поддержки/сопротивления. Были реализованы микросервисы для парсинга Telegram и Telegram-бот, предназначенный для интерактивной работы с пользователями, включая подтверждение входа, нотификации и обратную связь.

Пользовательский интерфейс (frontend) был реализован в виде SPA-приложения, позволяющего интерактивно работать с графиками, фильтрами, списками тикеров, предупреждениями и новостями. Через REST API frontend взаимодействует с backend и отображает только актуальные предобработанные данные.

Заключительным этапом разработки стала реализация виртуального хостинга – каждый микросервис был контейнеризирован с помощью Docker, а их взаимодействие организовано в единой среде посредством docker-compose. Это позволило не только упростить локальное тестирование и развёртывание, но и подготовить систему к публикации на удалённый хостинг SpaceWeb. Были описаны все Dockerfile, создан единый docker-compose.yaml, настроены зависимости, переменные окружения и проброс портов. Таким образом, была создана полностью автономная и готовая к эксплуатации архитектура, которую можно развернуть одной командой на любом сервере.

В качестве итогового вывода можно сказать, что все цели дипломного проекта были достигнуты. Разработано современное, гибкое и масштабируемое веб-приложение, которое решает важную прикладную задачу – предоставляет пользователям доступ к актуальной информации о фондовом рынке, предоставляет инструменты анализа и визуализации, облегчает принятие инвестиционных решений. Проект основывается на современных подходах к разработке, включая микросервисную архитектуру, асинхронную обработку данных, контейнеризацию, использование REST API и брокеров сообщений. Это делает приложение готовым к дальнейшему развитию, масштабированию и внедрению в реальные условия эксплуатации.

Таким образом, проделанная работа демонстрирует не только технические навыки в области разработки распределённых систем, но и системный подход к решению практической задачи анализа фондового рынка с учётом пользовательских потребностей и требований к надёжности, производительности и удобству эксплуатации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Aiogram. Python-фреймворк для разработки Telegram-ботов [Электронный ресурс] // Aiogram. – Режим доступа: https://docs.aiogram.dev.

2. amCharts. Библиотека JavaScript-графиков и диаграмм для визуализации данных [Электронный ресурс] // Библиотека графиков amCharts. – Режим доступа: https://www.amcharts.com.

3. Cassandra. Официальный источник технологии для базы данных Cassandra [Электронный ресурс] // Cassandra. – Режим доступа: https://cassandra.apache.org.

4. Figma. Официальный источник сервиса для создания интерфейсов figma [Электронный ресурс] // Figma. – Режим доступа: https://figma.com.

5. Metatrader5. Приложение для анализа рынка [Электронный ресурс] // Metatrader. – Режим доступа: mehttps://www.metatrader5.com.

6. Nestjs. Официальный источник технологии для backend Nestjs [Электронный ресурс] // Nestjs. – Режим доступа: https://nestjs.com.

7. SpaceWeb. Веб-приложение для аренды виртуальных машин [Электронный ресурс] // SpaceWeb. – Режим доступа: https://sweb.ru.

8. Telemetr Сервис для аналитики Telegram-каналов [Электронный ресурс] // Telemetr. – Режим доступа: https://telemetr.me.

9. Thinkorswim. Приложение для анализа рынка [Электронный ресурс] // Thinkorswim. – Режим доступа: – https://trade.thinkorswim.com.

10. Tinkoff API. Программный интерфейс для работы с продуктами Тинькофф Инвестиций [Электронный ресурс] // API Tinkoff. – Режим доступа: https://tinkoff.github.io/investAPI.

11. Tradingview. Приложение для анализа рынка [Электронный ресурс] // Tradingview. – Режим доступа: https://ru.tradingview.com.

12. Агальцов А.В. ACID [Электронный ресурс] // ACID. – Режим доступа: https://agaltsovav.ru.

13. Андреев Р.В. Монолиты, микросервисы или бессерверная архитектура: что выбрать для вашего проекта – большой обзор [Электронный ресурс] // Архитектура приложений. – Режим доступа: https://timeweb.cloud.

14. Афанасьева Ю.Р. Алгоритмическая торговля – что это, риски для инвестора [Электронный ресурс] // Алгоритмическая торговля. – Режим доступа: https://www.finam.ru.

15. Беляевскова О.А. Виды заявок на бирже [Электронный ресурс] // Заявки на бирже. – Режим доступа: https://beststocks.ru.

16. Бизли Д., Джонс Б. К. Python. Книга рецептов. – М.: ДМК Пресс, 2017. – 432 с.

17. Бирюков К.Е. Vue.js: что такое, как он устроен и чем отличается от React [Электронный ресурс] // Что такое Vue.js. – Режим доступа: https://skillbox.ru.

18. Бокова В.А. Опционы: стартовый гайд для начинающих инвесторов [Электронный ресурс] // Опционы. – Режим доступа: https://skillbox.ru.

19. Бхаргава А. Грокаем алгоритмы. Иллюстрированное пособие для программистов и любопытствующих. – СПб.: Питер, 2023. – 288 с.

20. Веселова Е.Н. Что такое биржевой ордер и как он работает [Электронный ресурс] // Биржевой ордер. – Режим доступа: https://gazprombank.investments.

21. Гаврилова Ю.А. О микросервисной архитектуре простыми словами - большой обзор [Электронный ресурс] // Микросервисная архитектура. – Режим доступа: https://skillbox.ru.

22. Голицина О.Л., Попов И.И., Максимов Н.В. Базы данных. – М.: ИНФРА-М, 2019 г. 400 с.

23. Голобурдин А. Типизированный Python для профессиональной разработки. – СПб.: Питер, 2023. – 320 с.

24. Грант К. CSS для профи. – СПб.: Питер, 2021. – 496 с.

25. Грозецкая Т.М. Как анализировать объёмы в трейдинге [Электронный ресурс] // Анализ объёмов. – Режим доступа: https://sbersova.ru.

26. Грушко М.А. Что такое форекс [Электронный ресурс] // Форкс. – Режим доступа: https://lenta.ru.

27. Даниленко А.Д. Как устроен фондовый рынок: полный гид для начинающего инвестора [Электронный ресурс] // Фондовый рынок. – Режим доступа: https://gazprombank.investments.

28. Даровская М.А. Как работает Docker: подробный гайд от техлида [Электронный ресурс] // Docker. – Режим доступа: https://skillbox.ru.

29. Демидова М.А. MongoDB: что это за СУБД, плюсы, минусы, подводные камни [Электронный ресурс] // MongoDB. – Режим доступа: https://skillbox.ru.

30. Демидова М.А. MySQL: что это, как устроена, чем хороша и как начать с ней работать [Электронный ресурс] // MySQL. – Режим доступа: https://skillbox.ru.

31. Демидова М.А. SQL и NoSQL: инь и ян в мире баз данных [Электронный ресурс] // SQL и NoSQl. – Режим доступа: https://skillbox.ru.

32. Дизайн приложения. Разработанный дизайн для веб-приложения в Figma [Электронный ресурс] // Макет в Figma. – Режим доступа: https://www.figma.com/design/ZWz5Er1UKKfGJKpztlxORj/StocksMonitoring?node-id=0-1&t=ScN5xAhBUpehgXzp-1.

33. Добрынина К.Е. Обзор универсального инструмента Figma [Электронный ресурс] // Figma. – Режим доступа: https://www.rbc.ru.

34. Дурманов С.В. Как создать и развернуть приложение на FastAPI: быстрый и простой деплой с Apps [Электронный ресурс] // Приложения на FasrAPI. – Режим доступа: https://timeweb.cloud.

35. Зверев Д.М. Фреймворк Flask: как он работает и зачем нужен [Электронный ресурс] // Flask. – Режим доступа: https://skillbox.ru.

36. Иваничев И.А. Для чего нужно SEO и что туда входит: разбор для новичков [Электронный ресурс] // SEO. – Режим доступа: https://skillbox.ru.

37. Каримов А.А. Информационные технологии в медицине. – Научно-технический центр, 2017 г. – 320 с.

38. Катерина В. и др. "Проектирование баз данных. СУБД Access" – М.: Диалог-МИФИ, 2016. – 352 с.

39. Когаловский М.Р. Энциклопедия технологий баз данных. – М.: Финансы и статистика, 2002 г. – 800 с.

40. Кодов А.М. Что такое Nuxt.js и как его использовать [Электронный ресурс] // Next.js. – Режим доступа: https://skillbox.ru.

41. Козлов И.А. "Современные технологии разработки баз данных" – СПб.: БХВ-Петербург, 2014. – 464 с.

42. Козлова О.Н. SQL Server 2012. Базы данных. Основы. – СПб.: БХВ-Петербург, 2014. – 272 с.

43. Краев В.С. Программирование баз данных в Delphi. – М.: Бином, 2015. – 512 страниц.

44. Кузнецова А.М. Django для новичков: что это за фреймворк и зачем его изучать [Электронный ресурс] // Django. – Режим доступа: https://skillbox.ru.

45. Кузнецова А.М. Next.js: гайд для новичков [Электронный ресурс] // SpaceWeb. – Next.js гайд: https://skillbox.ru.

46. Кучерявый. Е.А. Как написать одностраничное приложение на Angular.JS [Электронный ресурс] // Angular.JS. – Режим доступа: https://skillbox.ru.

47. Кучерявый. Е.А. Как работает .NET и зачем он нужен [Электронный ресурс] // .NET. – Режим доступа: https://skillbox.ru.

48. Кучерявый. Е.А. Тёмная тема: приёмы, которые помогут сделать качественный интерфейс [Электронный ресурс] // Почему тёмная тема хорошее решение. – Режим доступа: https://skillbox.ru.

49. Любанович Б. Простой Python. Современный стиль программирования. – М.: ДМК Пресс, 2021. – 384 с.

50. Маркин А. В. Построение запросов и программирование на SQL. – М.: Диалог-МИФИ, 2018. – 384 с.

51. Мартин Р. Чистый код. Создание, анализ и рефакторинг. – СПб.: Питер, 2021. – 464 с.

52. Мартишин С. В., Симонов В. В., Храпченко М. К. Базы данных. Работа с базами данных и файловыми системами. – М.: ИНФРА-М, 2019. - 235 с.

53. Мартишин С.Д., Симонов В.Д., Храпченко М.К. Проектирование, реализация баз данных СУБД SQL использованием MySQL Workbench. – СПб.: Форум, 2022. – 160 с.

54. Мартишин С.С., Симонов В.Д., Храпченко М. Д. Базы данных. Работа с распределенными базами данных и файловыми системами на примере MongoDB и HDFS. – М.: ИНФРА-М, 2020. – 245 с.

55. Мирзакаримова К.Е. Барные графики [Электронный ресурс] // Барные графики. – Режим доступа: https://journal.sovcombank.ru.

56. Михеев М.Е. Знакомимся с Redis [Электронный ресурс] // Redis. – Режим доступа: https://skillbox.ru.

57. Новиков Б.Л., Графеева Н. Н., Горшкова Е. Г. Основы технологий баз данных. Учебное пособие. – М.: ДМК-Пресс, 2020. – 582 с.

58. Осипов Д.Д. Технологии проектирования баз данных. – М.: ДМК-Пресс, 2019. – 498 с.

59. Остроух А.К., Суркова Н.Г. Проектирование информационных систем. – СПб.: Лань, 2021. – 164 с.

60. Пантелеев Е. В., Алыкова А. В. Алгоритмы сжатия данных без потерь. Учебное пособие для вузов. – СПб.: Лань, 2022. – 172 с.

61. Перри М. С. Искусство неизменяемой архитектуры: теория и практика управления данными в распределенных системах. – М.: ДМК-Пресс, 2022. – 388 с.

62. Персиваль Г. Python. Разработка на основе тестирования. – СПб.: Питер, 2022. – 624 с.

63. Персиваль Г., Грегори Б. Паттерны разработки на Python: TDD, DDD и событийно-ориентированная архитектура. – СПб.: Питер, 2022. – 448 с.

64. Погудин С.Н. Что такое трейлинг-стоп и как его применять? [Электронный ресурс] // трейдинг-стоп. – Режим доступа: https://www.finam.ru.

65. Полищук Ю.Е., Боровский А.А. Базы данных и их безопасность. Учебное пособие. – М.: ИНФРА-М, 2020. - 210 с.

66. Пономарева К.А. Что такое One-Cancels-the-Other (OCO) ордер? [Электронный ресурс] // One-Cancels-the-Other – Режим доступа: https://www.mexc.com.

67. Рамальо Л. Python. К вершинам мастерства. – М.: ДМК Пресс, 2021. – 720 с.

68. Руднева Е.Е. Рынок капиталов, фондовый рынок [Электронный ресурс] // Фондовый рынок. – Режим доступа: https://www.banki.ru.

69. Самаркина И.А. Японские свечи: как читать графики и понимать паттерны – инструкция для начинающих инвесторов [Электронный ресурс] // Японские свечи как инструмент в аналитике акций. – Режим доступа: https://gazprombank.investments.

70. Сафронов М.В. База данных MS SQL Server: что это, зачем нужна, как появилась и чем хороша [Электронный ресурс] // MS SQL Server. – Режим доступа: https://skillbox.ru.

71. Сафронов М.В. База данных Oracle DB: как она устроена и чем хороша [Электронный ресурс] // Orancle DB. – Режим доступа: https://skillbox.ru.

72. Семин А.Р. PostgreSQL: всё, что нужно знать для быстрого старта [Электронный ресурс] // PostgreSQL. – Режим доступа: https://skillbox.ru.

73. Симпсон К. ES6 и далее (Ты не знаешь js). – O'Reilly, 2015. – 217 с.

74. Симпсон К. this и прототипы объектов (Ты не знаешь js). – O'Reilly, 2014. – 158 с.

75. Симпсон К. Асинхронное программирование и производительность (Ты не знаешь js) // O'Reilly, 2015. – 169 с.

76. Симпсон К. Вверх и в путь (Ты не знаешь js). – O'Reilly, 2015. – 72 с.

77. Симпсон К. Область видимости и замыкания (Ты не знаешь js). – O'Reilly, 2014. – 83 с.

78. Симпсон К. Типы и грамматика (Ты не знаешь js). – O'Reilly, 2015. – 180 с.

79. Солабуто Н.В. Графики Ренко: как работать с этим индикатором [Электронный ресурс] // Графики Ренко. – Режим доступа: https://www.finam.ru.

80. Солабуто Н.Л. Индикатор Хейкен Аши: что это и как использовать [Электронный ресурс] // Индикатор Хейкен. – Режим доступа: https://www.finam.ru.

81. Стефанов С. JavaScript. Шаблоны. – М.: Символ-Плюс, 2011. – 272 с.

82. Тарасов С.С. СУБД для программиста. Базы данных изнутри. – СПб.: Солон-пресс, 2018. – 320 с.

83. Траск Э. Грокаем глубокое обучение. Научись строить нейросети. – СПб.: Питер, 2022. – 272 с.

84. Тукаев Т.М. React.js для новичков в программировании: что это, как устроен и зачем нужен [Электронный ресурс] // React.js. – Режим доступа: https://skillbox.ru.

85. Туманов В.В., Гайфуллин Б.В. И.SQL для хранения, обработки и анализа данных. - СПб.: Солон-пресс, 2021. – 172 с.

86. Флорин М. PostgreSQL. Администрирование и разработка. – М.: ДМК Пресс, 2017. – 544 с.

87. Фрэйн Б. HTML5 и CSS3. Разработка сайтов для любых браузеров и устройств. – СПб.: Питер, 2017. – 272 с.

88. Шитов В. Г. Основы проектирования баз данных. Учебное пособие. – М.: ИНФРА-М, 2023. – 236 с.

89. Эмеретли Х.А. Биржевой стакан: зачем он нужен и как его используют трейдеры [Электронный ресурс] // Биржевой стакан. – Режим доступа: https://gazprombank.investments.

90. Эмеретли Х.А. Для чего нужен метод скользящей средней и как его используют инвесторы [Электронный ресурс] // Метод скользящей средней. – Режим доступа: https://gazprombank.investments.

91. Эмеретли Х.А. Как читать графики на бирже: краткое руководство для начинающих инвесторов [Электронный ресурс] // Как читать графики на бирже. – Режим доступа: https://gazprombank.investments.

92. Яценко А.А. Запускаем первое веб-приложение на Spring Boot [Электронный ресурс] // приложения на Spring Boot. – Режим доступа: https://skillbox.ru.